高效大语言模型

第 10 章

推理特点 · Flash Attention · vLLM · 量化与投机解码

著者
邓杭　杨耀东

教材
《大语言模型：训练、精调与对齐》

出版
清华大学出版社



本章目录

10.1 **LLM推理特点分析** 推理两阶段与加速方案分类 第 3 页

10.2 **Flash Attention** Online-Softmax 与分块注意力 第 11 页

10.3 **vLLM 推理服务** 分页注意力与连续批处理 第 62 页

10.4 **量化压缩部署** SmoothQuant 与 AWQ 第 77 页

10.5 **投机解码** 块解码与投机解码 第 86 页

10.6 **小结** 四条加速路线各自的瓶颈与手段 第 97 页

阅读建议 先建立推理的性能瓶颈直觉（10.1），再读 Flash Attention、vLLM、量化与投机解码（10.2–10.5）。

10.1

LLM推理特点分析

推理无法像训练那样并行：模型逐词元生成，预填充与解码的计算特性截然不同，这决定了后面所有加速方法的着力点。

本节内容

10.1.1 LLM推理介绍

10.1.2 推理步骤

10.1.3 章节安排

自回归推理的过程

大语言模型大部分是自回归的纯解码器 Transformer。推理时，模型依赖之前的所有词元序列逐个预测下一个词元，因此无法像训练阶段一样并行处理。

$\boldsymbol{x}_{1:n}$

已有词元序列

→

前向计算

注意力读全部历史

复杂度 $O(n^2)$

→

x_{n+1}

只取最后一个位置的预测

→

$\boldsymbol{x}_{1:n+1}$

拼回序列，进入下一轮

- 训练时整条序列一次前向、各位置并行预测；推理时第 $t+1$ 个词元必须等第 t 个词元产生后才能计算。
- 序列长度 n 增长时，预测单个词元的时间复杂度为 $O(n^2)$，整体推理效率随之下降。
- 序列越长、批量越大，瓶颈越从“算不动”转向“搬不动”——显存带宽与 KV-Cache 占用成为主要约束。

推理加速的四类方案

内存优化

Flash Attention

对注意力分块计算，减少 HBM 与 SRAM 之间的数据存取次数，从而加速。同一套内核也适用于训练。

KV-Cache 管理

分页注意力 + 连续批处理

vLLM 以“虚拟页”管理随序列增长的 KV-Cache，并以连续批处理提高请求吞吐量与批量解码效率。

量化部署

SmoothQuant 与 AWQ

将参数与激活值量化为 8 位或 4 位，以低比特运算提升部署效率；典型形式为 W8A8、W4A8、W4A16。

并行解码

块并行解码与投机解码

以低成本方式猜测多个词元，再用目标模型并行验证：块解码增加输出头，投机解码借助小模型。

怎么选 四类方案分别作用于访存、显存管理、数值精度与解码步数，可针对场景组合使用：访存受限先上 Flash Attention，显存受限先上分页注意力，算力受限先上量化。

系统层与计算层的优化

优化可以分两层来看：系统层解决“请求怎么排队、缓存怎么放”，计算层解决“算子怎么算、精度怎么选”。

系统层

服务框架与调度

vLLM、SGLang 侧重提升服务效率；Mooncake 专注预填充-解码分离下的 KV-Cache 管理。

计算层

算子与精度

算子优化（如内核融合）减少访存与启动开销；混合精度以低精度存储、高精度计算兼顾速度与数值稳定。

通信重叠

通信密集型任务设计计算-通信重叠流程，减少等待延迟。

按场景组合

优化依据 Transformer 的自回归特性，可按场景自由组合。

通信与计算均衡

分布式集群上还需权衡两者开销，避免一侧空转。

提示 本章沿“访存 → 显存 → 精度 → 解码步数”的顺序展开，正好对应上面四类方案。

预填充阶段

先对整条输入序列做一次前向计算，填充每一层注意力模块的 KV-Cache，这一步称为预填充（Prefill）。

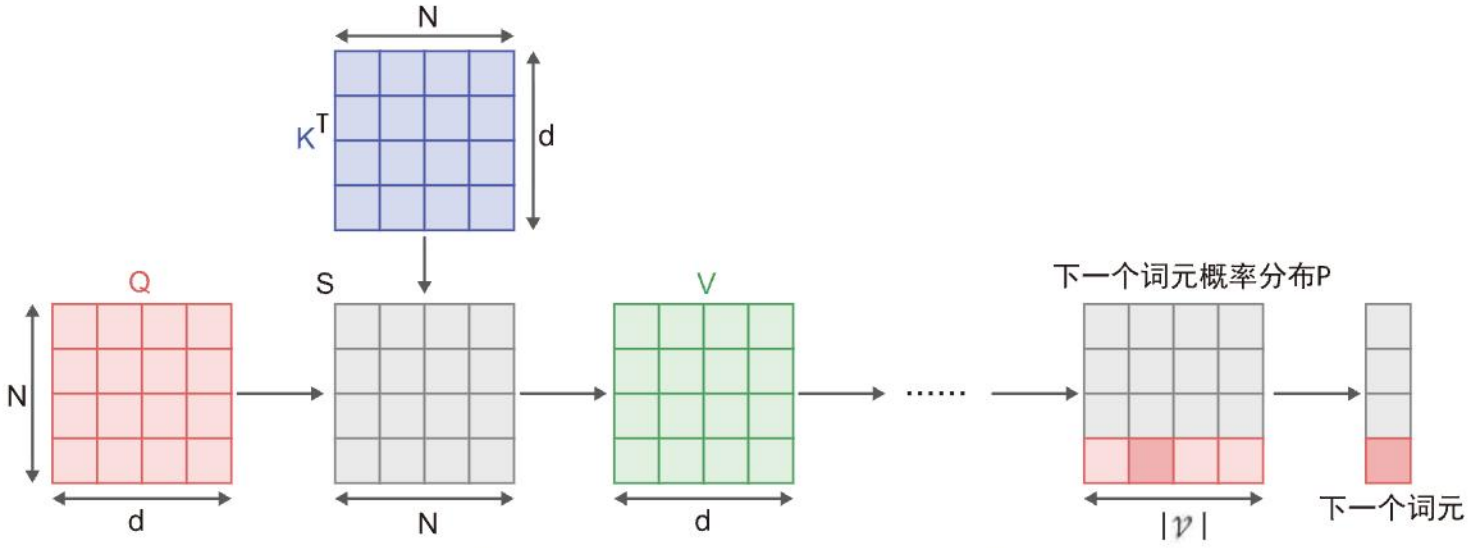


图 10.1　LLM 推理过程的预填充阶段

一次前向

整条输入序列一次算完，逐层填满 KV-Cache。

取最后一个位置

前向会预测多个位置，只取最后一个位置的输出。

计算密集

密集矩阵运算，高效算注意力是加速关键。

图源 《大语言模型：训练、精调与对齐》图 10.1

解码阶段

解码阶段只输入最新的一个词元，借助历史的 KV-Cache 完成注意力计算，从而预测下一个词元的概率分布。

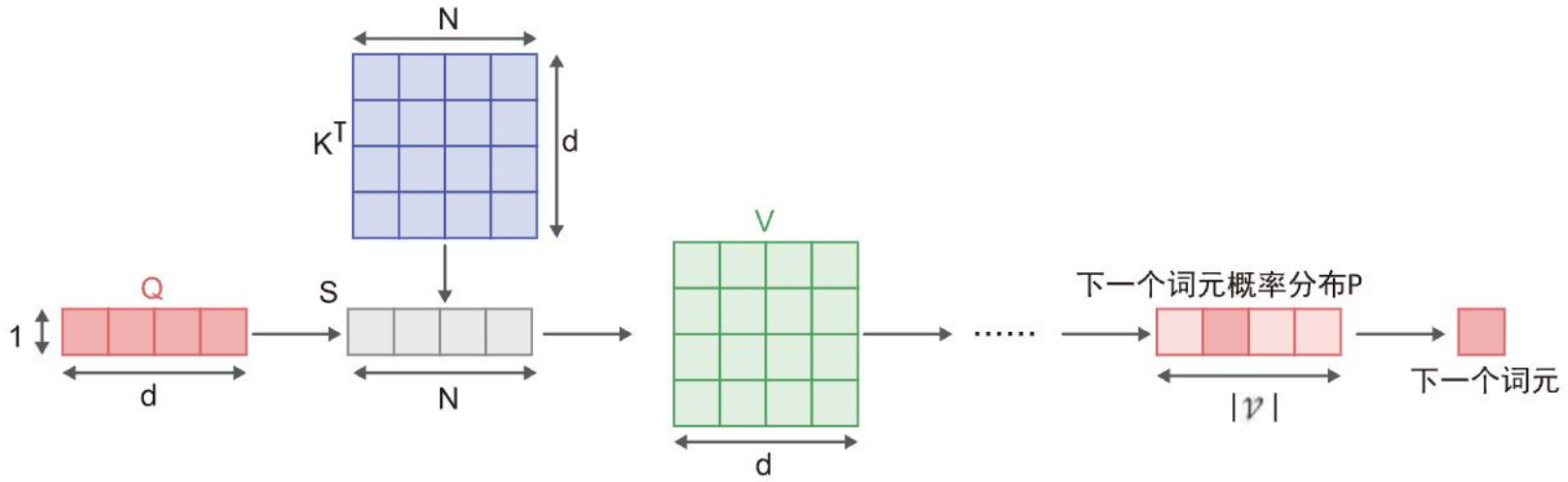


图 10.2　LLM 推理过程的解码阶段

一次一个词元

一个查询向量与历史全部键值向量做注意力计算。

串行推进

每步只生成一个词元，步与步之间无法并行。

靠批量提并行度

同时处理更大批量的请求，而不是把一条序列切开。

图源 《大语言模型：训练、精调与对齐》图 10.2

两个阶段的指标

对比项	预填充（Prefill）	解码（Decoding）
输入	整条输入序列	最新一个词元（配合 KV-Cache）
计算特性	计算密集，矩阵运算充分并行	存储密集，频繁的串行小计算
关键指标	首词元预测时间 TTFT	每秒生成词元数（吞吐量）
加速手段	Flash Attention、上下文并行	增大解码批量、连续批处理

- TTFT 越大，实时对话场景中用户等待越久；吞吐量越大，推理系统的效率越高。
- **预填充-解码分离（PD 分离）**：把两个阶段放到性能特性不同的机器上处理。
- **分块预填充**：把解码任务并入填充批次，通过“搭便车”式的前向计算提高整体吞吐。

本章结构

10.2 Flash Attention

加速预填充阶段的注意力：Online-Softmax 分块，避免 HBM 与 SRAM 反复交换中间矩阵；同样适用于训练。

10.3 vLLM 推理服务

以分页注意力管理 KV-Cache，提高显存利用率与批量解码吞吐量。

10.4 量化压缩部署

SmoothQuant 与 AWQ：在低比特精度下执行权重与激活值的计算。

10.5 投机解码

自回归模型特有的并行解码技术，让模型一次生成多个词元。

长序列的分布式注意力计算见上下文并行一节：切分序列可降低显存需求并加速前向，直接降低 TTFT。

10.2

Flash Attention

注意力真正的瓶颈不是算力，而是显存带宽。把 Softmax 改写成可增量聚合的形式，就能分块计算注意力而不落地中间矩阵。

本节内容

10.2.1 Online Softmax

10.2.2 Flash Attention

10.2.3 Flash Attention 反向

10.2.4 Flash Attention-V2

10.2.5 Flash Attention-V2 反向

推理的性能瓶颈

注意力的计算与存储复杂度随序列长度平方增长。部分加速算法通过改进注意力机制减少计算量，却未能显著提速——Flash Attention 发现长上下文注意力的瓶颈在于内存墙（Memory-Bound）。

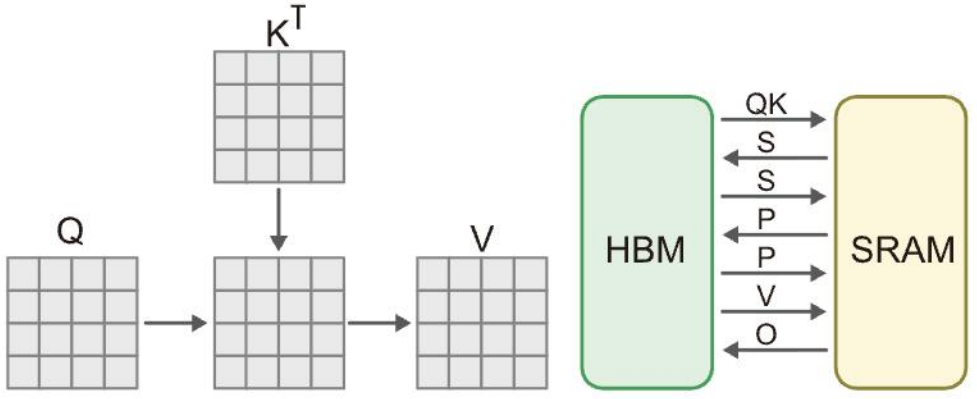


图 10.3(a)　标准注意力

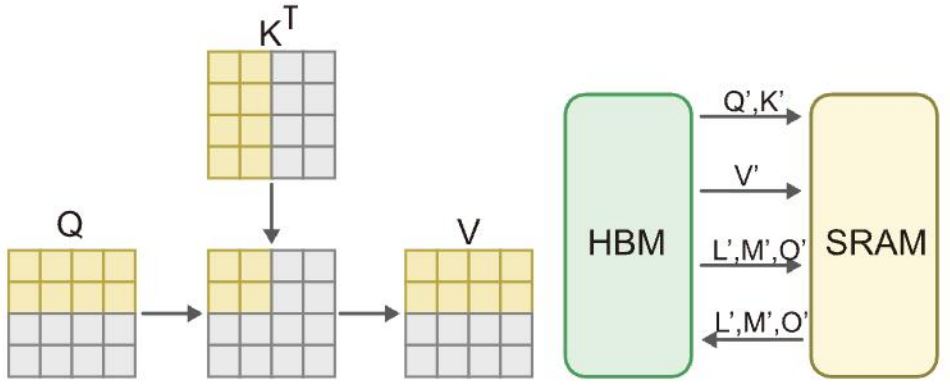


图 10.3(b)　Flash Attention

图 10.3　注意力计算过程中 HBM 与 SRAM 的数据交换情况

HBM 是 GPU 的高带宽、大容量片外显存；SRAM 是高速缓存层。$\boldsymbol{QK}$ 为查询与键矩阵，$\boldsymbol{S}$ 为注意力分数，$\boldsymbol{P}$ 为注意力概率，$\boldsymbol{V}$ 为值矩阵，$\boldsymbol{O}$ 为输出；$\boldsymbol{L}'$、$\boldsymbol{M}'$ 为分块保留的辅助值，$\boldsymbol{O}'$ 为局部注意力输出。

图源 《大语言模型：训练、精调与对齐》图 10.3

标准注意力的访存开销

$$\boldsymbol{O}_i = \boldsymbol{P}_i \boldsymbol{V} = \texttt{Softmax}\left(\frac{\boldsymbol{Q}_i \boldsymbol{K}^T}{\sqrt{d}}\right)\boldsymbol{V} \tag{10-1}$$

$\boldsymbol{Q}_i \in \mathbb{R}^{1\times d}$，$\boldsymbol{K} \in \mathbb{R}^{N\times d}$，$N$ 为序列词元数量，d 为特征维度。

- 计算复杂度为 $O(N^2 d)$，且注意力分数需存储于 HBM，产生 $O(N^2)$ 读写操作。
- 若 SRAM 无法一次性完成注意力计算，就要多次在 HBM 与 SRAM 之间交换中间数据（图 10.3a）。
- Flash Attention 改为分块计算局部注意力，减少交换次数（图 10.3b）。
- XFormers 与 Rabe 等的框架同样采用分块思想，目标是降低显存占用，但未减少高速缓存的交换次数。

分块 Softmax 的问题

$$\boldsymbol{P}_i = \texttt{Softmax}\left(\frac{\boldsymbol{Q}_i\boldsymbol{K}^T}{\sqrt{d}}\right) = \texttt{Softmax}\left(\left[\frac{\boldsymbol{Q}_i\boldsymbol{K}^T_{:N/2}}{\sqrt{d}}, \frac{\boldsymbol{Q}_i\boldsymbol{K}^T_{N/2:}}{\sqrt{d}}\right]\right) \tag{10-2}$$

$$\neq \left[\texttt{Softmax}\left(\frac{\boldsymbol{Q}_i\boldsymbol{K}^T_{:N/2}}{\sqrt{d}}\right), \texttt{Softmax}\left(\frac{\boldsymbol{Q}_i\boldsymbol{K}^T_{N/2:}}{\sqrt{d}}\right)\right] \tag{10-3}$$

$\boldsymbol{K} \to [\boldsymbol{K}_{:N/2}; \boldsymbol{K}_{N/2:}]$ 为分块矩阵：先拼接再归一化，与先分别归一化再拼接，结果不同。

只算块内概率

局部归一化的分母只是块内指数和，无法得到全局注意力权重。

先算全部分数

需要把 $N \times N$ 的分数矩阵写回 HBM，交换次数不降反升。

加速的目标：分块计算注意力分数，在避免额外 HBM-SRAM 中间值交换的前提下，保证全局分数概率的准确性。

Safe-Softmax 的全局统计

$$\tilde{x}_i = \frac{\mathrm{e}^{x_i - m}}{\sum_{j=1}^{N} \mathrm{e}^{x_j - m}} \tag{10-4}$$

为防止 Softmax 数值上溢而减去全局最大值 $m = \max(\boldsymbol{x})$。它依赖全局指数和 $\sum_{j=1}^{N} \mathrm{e}^{x_j - m}$ 与全局最大值 m，因此分块计算前需先统计全局信息。

第 1 遍

遍历数组求全局最大值

$$m = \max(\boldsymbol{x})$$

→

第 2 遍

求全局指数和

$$l = \sum_{j=1}^{N} \mathrm{e}^{x_j - m}$$

→

第 3 遍

逐元素计算概率

$$\tilde{x}_i = \mathrm{e}^{x_i - m} / l$$

Safe-Softmax 需要三次遍历。要把它装进分块循环，就得把“先统计全局”改成“边算边更新全局”。这正是接下来三步推导的出发点：

- **Online Softmax**：把非线性 Softmax 的分块与聚合写成递推。
- **Flash Attention**：基于 Online-Softmax 推导单次循环的注意力计算。
- **Flash Attention-V2**：优化循环排布，进一步减少访存开销。

Softmax 的定义

$$\tilde{x}_i = \frac{\mathrm{e}^{x_i}}{\sum_j^N \mathrm{e}^{x_j}} \tag{10-5}$$

$\tilde{x}_i \in [0,1]$ 且 $\sum_{i=1}^{N} \tilde{x}_i = 1$，即输出是一个概率分布。

- 分子只与 x_i 有关，分母是全部元素指数之和。
- 分母一旦变化，所有元素的结果都要跟着变——这是分块困难的根源。

代码 6　Softmax 代码（完整；结果注释折为两行）

```
X = torch.tensor([-0.3, 0.2, 0.5, 0.7, 0.1, 0.8])
X_exp_sum = X.exp().sum()
X_softmax = torch.exp(X) / X_exp_sum
% print(X_softmax)
% 结果: tensor([0.0827, 0.1364, 0.1841,
%                0.2249, 0.1234, 0.2485])
```

依据 《大语言模型：训练、精调与对齐》第 10.2.1 节（式 (10-5)、代码 6）

Safe-Softmax

$$\tilde{x}_i = \frac{\mathrm{e}^{x_i - \max(\boldsymbol{x})}}{\sum_{j=1}^{N} \mathrm{e}^{x_j - \max(\boldsymbol{x})}} \tag{10-6}$$

元素值较大时，直接计算 e^{x_i} 会数值上溢；把所有元素减去全局最大值后，指数项的最大值为 1。

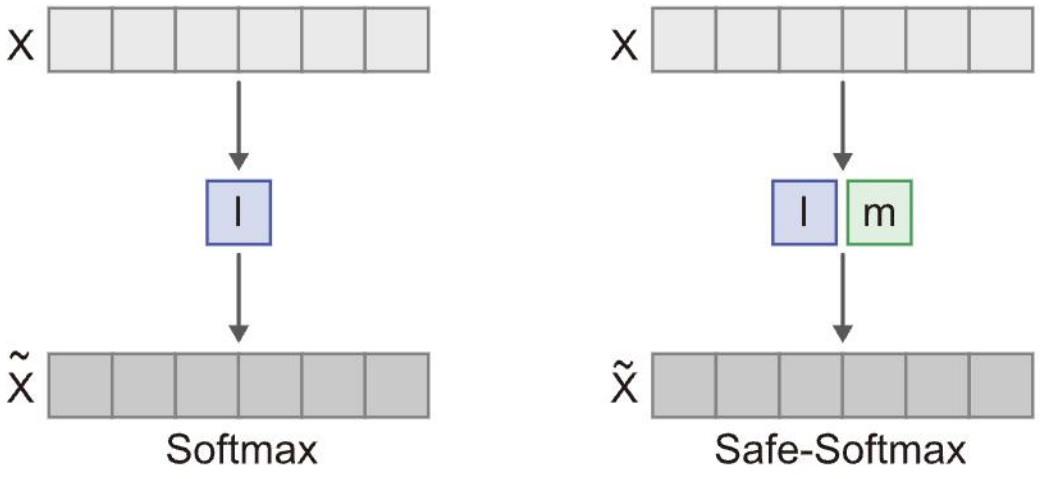


图 10.4(a)　Softmax　　图 10.4(b)　Safe-Softmax

图 10.4　Safe-Softmax 计算流程

图源 《大语言模型：训练、精调与对齐》图 10.4

从 Safe-Softmax 到增量更新

$$\tilde{x}_i = \frac{\mathrm{e}^{x_i - \max(\boldsymbol{x})}}{\sum_{j=1}^{N} \mathrm{e}^{x_j - \max(\boldsymbol{x})}} = \frac{\mathrm{e}^{x_i - M}}{\sum_{j=1}^{N} \mathrm{e}^{x_j - M}} \tag{10-7}$$

$$= \frac{\mathrm{e}^{x_i}/\mathrm{e}^{M}}{\sum_{j=1}^{N} \mathrm{e}^{x_j}/\mathrm{e}^{M}} = \frac{\mathrm{e}^{x_i}}{\sum_{j=1}^{N} \mathrm{e}^{x_j}} \tag{10-8}$$

$$l_N = \sum_{j=1}^{N} \mathrm{e}^{x_j - \max(\boldsymbol{x}_{:N})} = \sum_{j=1}^{N} \mathrm{e}^{x_j - m_N} \tag{10-9}$$

$$m_{N+1} = \max(m_N, x_{N+1}) \tag{10-10}$$

$$l_{N+1} = \sum_{j=1}^{N+1} \mathrm{e}^{x_j - m_{N+1}} \tag{10-11}$$

$$= \left(\sum_{j=1}^{N} \mathrm{e}^{x_j - m_{N+1}}\right) + \mathrm{e}^{x_{N+1} - m_{N+1}} \tag{10-12}$$

其中 $M = \max(\boldsymbol{x})$。式 (10-7) 把 $\mathrm{e}^{x_i - M}$ 拆为 $\mathrm{e}^{x_i}/\mathrm{e}^{M}$，式 (10-8) 分子分母同乘 e^{M} 回到式 (10-5)；式 (10-11) 按"旧元素 + 新元素"拆开得式 (10-12)。

为什么可以减常数　减去常数不改变 Softmax 的结果，只改变数值范围，因此可以放心用 M 来稳定计算。代码 7 正是这三步：取最大值、按新基准求指数和、再逐元素归一化。

```
X_max  = X.max()
X_sum  = torch.exp(X - X_max).sum()
X_safe = torch.exp(X - X_max) / X_sum
```

为什么还不能直接递推　式 (10-12) 中旧元素仍带着新基准 m_{N+1}，代不回旧的 l_N。下一步的关键是把 $\mathrm{e}^{x_j - m_{N+1}}$ 改写成 $\mathrm{e}^{x_j - m_N}\mathrm{e}^{m_N - m_{N+1}}$：同一项拆成"旧基准下的值"与"两个基准之间的修正因子"两部分。

依据 《大语言模型：训练、精调与对齐》第 10.2.1 节（式 (10-7)–(10-12)，代码 7）

指数和的基准变换

$$l_{N+1} = \left(\sum_{j=1}^{N} \mathrm{e}^{x_j - m_N} \mathrm{e}^{m_N - m_{N+1}}\right) + \mathrm{e}^{x_{N+1} - m_{N+1}} \tag{10-13}$$

$$= \left(\sum_{j=1}^{N} \mathrm{e}^{x_j - m_N}\right) \mathrm{e}^{m_N - m_{N+1}} + \mathrm{e}^{x_{N+1} - m_{N+1}} \tag{10-14}$$

$$= l_N \left(\mathrm{e}^{m_N - m_{N+1}}\right) + \mathrm{e}^{x_{N+1} - m_{N+1}} \tag{10-15}$$

式 (10-13) 用 $\mathrm{e}^{x_j - m_N}\mathrm{e}^{m_N - m_{N+1}} = \mathrm{e}^{x_j - m_{N+1}}$ 统一到新基准；式 (10-14) 把与 j 无关的因子 $\mathrm{e}^{m_N - m_{N+1}}$ 提到求和号之外；式 (10-15) 括号内正是式 (10-9) 的 l_N。

第 1 项

旧指数和 l_N 乘以修正因子 $\mathrm{e}^{m_N - m_{N+1}}$。

第 2 项

新元素在新基准下的指数值 $\mathrm{e}^{x_{N+1} - m_{N+1}}$。

指数和的递推式

$$l_{N+1} = l_N \, \mathrm{e}^{m_N - m_{N+1}} + \mathrm{e}^{x_{N+1} - m_{N+1}}$$

$$\tilde{x}_i = \frac{\mathrm{e}^{x_i - m_{N+1}}}{l_{N+1}} \qquad (10\text{-}16)$$

第一行即上页的式 (10-15)：只依赖旧的 l_N, m_N 与新元素 x_{N+1}。第二行式 (10-16) 用更新后的全局量给出全部元素的概率。两者合起来即完整的 Online-Softmax。

- 最大值没变时 $m_N = m_{N+1}$，修正因子为 1，直接累加即可。
- 最大值被刷新时，旧的指数和按 $\mathrm{e}^{m_N - m_{N+1}}$ 等比缩放，与新基准对齐。

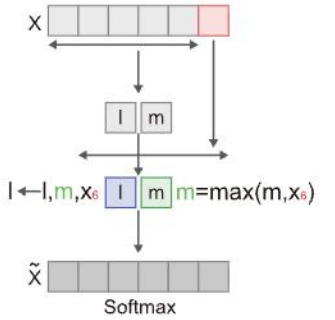


图 10.5(a)　逐元素更新

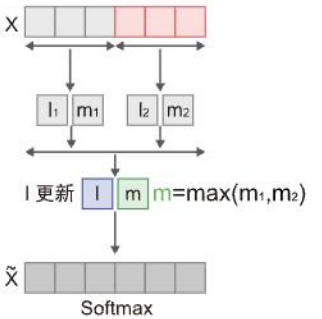


图 10.5(b)　逐块更新

图 10.5　Online Softmax 计算示例图

图源 《大语言模型：训练、精调与对齐》图 10.5(a)(b)；式 (10-15)、(10-16)

增量式 Online-Softmax 代码

代码 8　Online-Softmax 代码

```
1  X = torch.tensor([-0.3, 0.2, 0.5, 0.7, 0.1, 0.8])
2  X_pre = X[:-1]          # 前 5 个元素
3  X_max_pre = X_pre.max()
4  X_sum_pre = torch.exp(X_pre - X_max_pre).sum()
5  X_max_cur = torch.max(X_max_pre, X[-1])
6  X_sum_cur = X_sum_pre * torch.exp(X_max_pre - X_max_cur) \
7              + torch.exp(X[-1] - X_max_cur)
8  X_online_softmax = torch.exp(X - X_max_cur) / X_sum_cur
```

- 第 3、4 行：前 N 个元素的 m_N、l_N。
- 第 5 行：式 (10-10)，新元素可能刷新全局最大值。
- 第 6、7 行：式 (10-15) 的两项，先缩放旧和，再加新项。
- 第 8 行：式 (10-16)，用新的 m、l 归一化全部元素。

输出与直接调用 Softmax 完全一致，仍为 6 个概率值。

依据 《大语言模型：训练、精调与对齐》第 10.2.1 节（代码 8）

块状更新

$$m_1 = \max(\boldsymbol{x}_{:N}), \qquad m_2 = \max(\boldsymbol{x}_{N+1:2N}) \tag{10-17}$$

$$l_1 = \sum_{j=1}^{N} \mathrm{e}^{x_j - m_1}, \qquad l_2 = \sum_{j=N+1}^{2N} \mathrm{e}^{x_j - m_2} \tag{10-18}$$

新增数据块 $\boldsymbol{x}_{N+1:2N}$ 时，各块的局部最大值与局部指数和可以独立、并行地计算；注意 l_1 与 l_2 基于不同的局部最大值。

$$\tilde{m} = \max(\boldsymbol{x}_{:2N}) = \max\left(\max(\boldsymbol{x}_{:N}), \max(\boldsymbol{x}_{N+1:2N})\right) = \max(m_1, m_2) \tag{10-19}$$

$$\tilde{l} \neq l_1 + l_2 \tag{10-20}$$

全局最大值取两块最大值的较大者；但全局指数和不能直接相加——两个局部和用了不同的基准。

要把 l_1、l_2 相加，必须先把它们换到同一个基准 $\tilde{m}$ 上。

全局指数和

$$\tilde{l} = \sum_{j=1}^{2N} \mathrm{e}^{x_j - \max(\boldsymbol{x}_{:2N})} \tag{10-21}$$

$$= \left(\sum_{j=1}^{N} \mathrm{e}^{x_j - \tilde{m}} \right) + \left(\sum_{j=N+1}^{2N} \mathrm{e}^{x_j - \tilde{m}} \right) \tag{10-22}$$

$$= \left(\sum_{j=1}^{N} \mathrm{e}^{x_j - m_1} \right) \mathrm{e}^{m_1 - \tilde{m}} + \left(\sum_{j=N+1}^{2N} \mathrm{e}^{x_j - m_2} \right) \mathrm{e}^{m_2 - \tilde{m}} \tag{10-23}$$

$$= l_1 \mathrm{e}^{m_1 - \tilde{m}} + l_2 \mathrm{e}^{m_2 - \tilde{m}} \tag{10-24}$$

式 (10-22) 按块拆开求和；式 (10-23) 用 $\mathrm{e}^{x_j - \tilde{m}} = \mathrm{e}^{x_j - m_i} \mathrm{e}^{m_i - \tilde{m}}$ 把每块换到全局基准；式 (10-24) 代回局部和的定义。这就是式 (10-20) 不能直接相加的修正式。

$$\tilde{x}_i = \frac{\mathrm{e}^{x_i - \tilde{m}}}{\tilde{l}} \tag{10-25}$$

最终概率仍是一次归一化，只是分母换成聚合后的 $\tilde{l}$。合并后的 $\tilde{l}$ 与两个局部和 l_1, l_2 之间差着两个修正因子 $\mathrm{e}^{m_i - \tilde{m}}$，这正是式 (10-20) 不成立的原因。

两块合并的代码

代码 9　块状 Online-Softmax 代码

```
1  X_block = torch.split(X, split_size_or_sections = 3)
2  m_0 = X_block[0].max()
3  l_0 = torch.exp(X_block[0] - m_0).sum()
4  m_1 = X_block[1].max()
5  l_1 = torch.exp(X_block[1] - m_1).sum()
6
7  m_new = torch.max(m_0, m_1)
8  l_new = l_0 * torch.exp(m_0 - m_new) \
9        + l_1 * torch.exp(m_1 - m_new)
10 X_result = torch.exp(X - m_new) / l_new
```

- 第 2–5 行：两块各自求 m_i、l_i，彼此无关，可并行（式 (10-17)、(10-18)）。
- 第 7 行：式 (10-19) 的 $\tilde{m}$。
- 第 8、9 行：式 (10-24)，每块的指数和乘上自己的修正因子后相加。
- 第 10 行：式 (10-25) 归一化。

结果与标准 Softmax 相同，但峰值显存只需容纳一个块的指数值。

多块合并

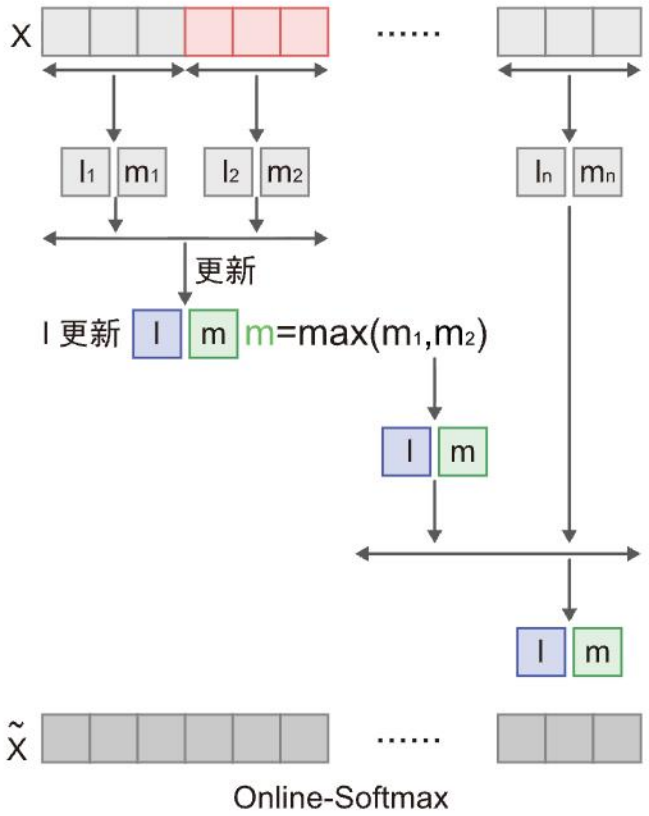


图 10.6　多块 Online-Softmax 更新示意图

各块的 m_i 与 l_i 并行计算，再通过循环迭代聚合全局状态。

代码 10　多块状 Online-Softmax 代码（节选：省略了 import 与结果打印）

```
X_blocks = torch.split(X, split_size_or_sections = 2)
M_new = torch.tensor([-float('inf')])
L_new = torch.tensor([0.0])
M_list, L_list = [], []
for i in range(len(X_blocks)):
    m = torch.max(X_blocks[i])
    l = torch.exp(X_blocks[i] - m).sum()
    M_list.append(m); L_list.append(l)
for i in range(len(X_blocks)):
    M_old, L_old = M_new, L_new
    M_new = torch.max(M_new, M_list[i])
    L_new = L_old * torch.exp(M_old - M_new) \
          + L_list[i] * torch.exp(M_list[i] - M_new)
```

第一个循环求各块局部量（无依赖，可并行）；第二个循环逐块把局部量并入全局。状态初值取 $-\infty$ 与 0，第一块并入时修正因子自然退化为 1。

图源 《大语言模型：训练、精调与对齐》图 10.6；依据代码 10

增量式 Online-Softmax

$$m^{\text{new}} = \max\left(m, \max(\boldsymbol{x}_{N+1:2N})\right) \tag{10-26}$$

$$l^{\text{new}} = l\,\mathrm{e}^{m-m^{\text{new}}} + \left(\sum_{j=N+1}^{2N} \mathrm{e}^{x_j - m^{\text{new}}}\right) \tag{10-27}$$

$$m \leftarrow m^{\text{new}}, \qquad l \leftarrow l^{\text{new}} \tag{10-28}$$

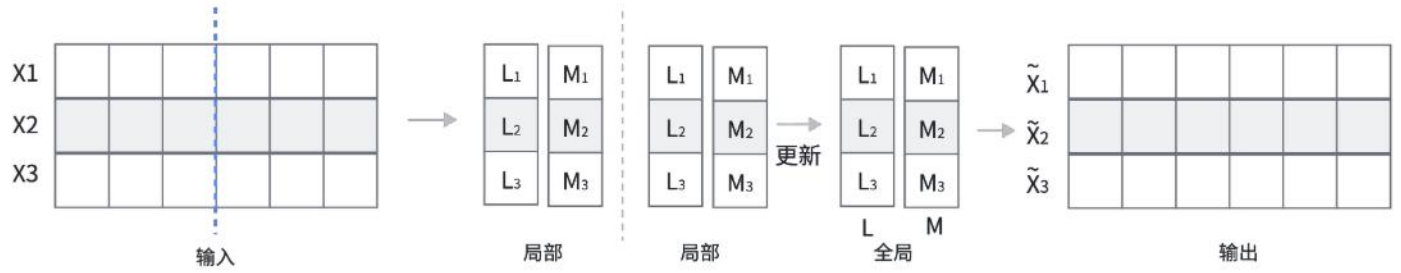
初始化 $m=-\infty$、$l=0$，然后对每个新块重复式 (10-26)–(10-28)：先刷新全局最大值，再把旧和缩放到新基准并累加本块贡献，最后写回状态。

式 (10-27) 与式 (10-15) 同构：单元素换成整块，$\mathrm{e}^{x_{N+1}-m_{N+1}}$ 换成块内指数和。这样只需一次遍历。

代码 11　多块状 Online-Softmax 简化代码

```
for i in range(len(X_blocks)):
    M_old, L_old = M_new, L_new
    M = torch.max(X_blocks[i])
    M_new = torch.max(M, M_old)
    L_new = L_old * torch.exp(M_old - M_new) \
          + torch.exp(X_blocks[i] - M_new).sum()
```

批量 Online-Softmax



$$\boldsymbol{M} = (-\infty) \in \mathbb{R}^{B\times 1}$$
$$\boldsymbol{L} = (\mathbf{0}) \in \mathbb{R}^{B\times 1}$$

每行独立初始化全局最大值与指数和。

图 10.7　批量化 Online-Softmax 示意图

- $\boldsymbol{X} \in \mathbb{R}^{B\times N}$ 按列切成 $k = \lceil N/d \rceil$ 块；每块用 rowmax、rowsum 得到 $\boldsymbol{M}'$、$\boldsymbol{L}_{ij}$。
- $\boldsymbol{L}^{\text{new}} = \boldsymbol{L}\exp(\boldsymbol{M} - \boldsymbol{M}^{\text{new}}) + \text{rowsum}\left(\exp(\boldsymbol{X}_i - \boldsymbol{M}^{\text{new}})\right)$，最终 $\tilde{\boldsymbol{X}} = \text{diag}(\boldsymbol{L})^{-1}\exp(\boldsymbol{X} - \boldsymbol{M})$。

$\exp(\boldsymbol{X})$ 与 $\mathrm{e}^{\boldsymbol{X}}$ 均为逐元素指数算子，不是矩阵指数。行最大值与行指数和沿行方向独立，因此 B 行可以完全并行。

图源 《大语言模型：训练、精调与对齐》图 10.7；依据该节算法与式 (10-26)–(10-28)

批量块状 Online-Softmax 代码

代码 12　批量块状 Online-Softmax 代码

```
X_batch = torch.randn(4, 6)
X_blocks = torch.split(X_batch, 2, dim=1)
b, _ = X_batch.shape
M_new = torch.ones(b, 1) * torch.tensor([-float('inf')])
L_new = torch.zeros(b, 1)
for X_block in X_blocks:
    M_old, L_old = M_new, L_new
    M, _ = torch.max(X_block, dim = 1, keepdim = True)
    M_new = torch.maximum(M, M_old)
    L_new = L_old * torch.exp(M_old - M_new) \
          + torch.exp(X_block - M_new).sum(dim = 1, keepdim = True)
X_result = torch.exp(X_batch - M_new) / L_new
```

- `dim=1, keepdim=True` 保证归约方向是“行内”，结果形状为 $B \times 1$。
- 逐块更新 $\boldsymbol{M}$、$\boldsymbol{L}$ 的写法与单行完全一致，只是全部换成张量运算。
- `split(dim=1)` 沿列切块，对应注意力矩阵按 K 的块切分。

依据 《大语言模型：训练、精调与对齐》第 10.2.1 节（代码 12）

批量块状 Online-Softmax 算法

算法 16 批量块状 Online-Softmax

输入： $\boldsymbol{X} \in \mathbb{R}^{B\times N}$，块大小 d
输出： $\tilde{\boldsymbol{X}} \in \mathbb{R}^{B\times N}$

1: 块大小为 d，列切分 $k = \lceil N/d \rceil$ 块 $\boldsymbol{X}_1, \boldsymbol{X}_2, \cdots, \boldsymbol{X}_k \in \mathbb{R}^{B\times d}$
2: 初始化 $\boldsymbol{M} = (-\infty) \in \mathbb{R}^{B\times 1}$, $\boldsymbol{L} = (\boldsymbol{0}) \in \mathbb{R}^{B\times 1}$
3: **for** $i = 1, 2, ..., k$ **do**
4: $\quad \boldsymbol{M}' = \mathrm{rowmax}(\boldsymbol{X}_i) \in \mathbb{R}^{B\times 1}$
5: $\quad \boldsymbol{M}^{\mathrm{new}} = \mathrm{rowmax}(\boldsymbol{M}, \boldsymbol{M}') \in \mathbb{R}^{B\times 1}$
6: $\quad \boldsymbol{L}^{\mathrm{new}} = \boldsymbol{L}\exp(\boldsymbol{M} - \boldsymbol{M}^{\mathrm{new}}) + \mathrm{rowsum}(\exp(\boldsymbol{X}_i - \boldsymbol{M}^{\mathrm{new}})$
7: $\quad \boldsymbol{L} \leftarrow \boldsymbol{L}^{\mathrm{new}}$, $\boldsymbol{M} \leftarrow \boldsymbol{M}^{\mathrm{new}}$
8: **end for**
9: $\tilde{\boldsymbol{X}} = \mathrm{diag}(\boldsymbol{L})^{-1}\exp(\boldsymbol{X} - \boldsymbol{M})$

算法 16　批量块状 Online-Softmax

- 每一行独立维护 $(\boldsymbol{M}, \boldsymbol{L})$，行与行互不影响，因此可以整批并行。
- 第 9 步在循环结束后才除以 $\boldsymbol{L}$；循环体内不做归一化。
- 第 6 步把旧的指数和乘上修正因子 $\mathrm{e}^{\boldsymbol{M}-\boldsymbol{M}^{\mathrm{new}}}$，再与新块的部分和相加。
- 这正是 Flash Attention 在 SRAM 内逐块累积所需的原语。

Online-Softmax 的收益

至此完成 Online-Softmax 的多种形式推导，其核心是指数和 $\sum_{j=1}^{N} \mathrm{e}^{x_j - m}$ 在增量环境下的迭代更新。

收益 1

并行计算

各数据块内的最大值和指数和互不依赖，可以并行计算。

收益 2

计算效率

传统方法需三次遍历（最大值 / 指数和 / 归一化），Online-Softmax 仅需两次遍历。

收益 3

内存优化

分块处理避免存储完整注意力矩阵，显著降低内存占用。

> **要点** 这三条正是把 Softmax 搬进注意力内核所需的全部条件：可并行、少遍历、不落地。下一节据此改写注意力。

注意力计算的四步

先给出标准注意力计算形式。为便于推导，暂忽略缩放因子 $\sqrt{d}$。

$$\boldsymbol{S} = \boldsymbol{Q}\boldsymbol{K}^T \tag{10-29}$$

$$M_i = \max(\boldsymbol{S}_i) \tag{10-30}$$

$$L_i = \sum_{j=1}^{N} \mathrm{e}^{S_{ij} - M_i} \tag{10-31}$$

$$\boldsymbol{P}_i = \texttt{Softmax}(\boldsymbol{S}_i) = \frac{\mathrm{e}^{\boldsymbol{S}_i - M_i}}{L_i} \tag{10-32}$$

$$\boldsymbol{O} = \boldsymbol{P}\boldsymbol{V} \tag{10-33}$$

$\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V} \in \mathbb{R}^{N\times d}$，$\boldsymbol{S} \in \mathbb{R}^{N\times N}$；$\boldsymbol{S}_i$ 为第 i 个查询的行分数，$\boldsymbol{M} \in \mathbb{R}^{N\times 1}$ 为各行最大值，$\boldsymbol{L} \in \mathbb{R}^{N\times 1}$ 为各行指数和，$\boldsymbol{P} \in \mathbb{R}^{N\times N}$，$\boldsymbol{O} \in \mathbb{R}^{N\times d}$。

$\boldsymbol{S}$ 与 $\boldsymbol{P}$ 都是 $N \times N$ 矩阵。标准实现要把它们写回 HBM，这正是分块计算要避免的。

逐列扫描的计算

第一个循环（$j = 1, 2, \cdots, N$）：累计 M_j、L_j。

$$S_{ij} \leftarrow \boldsymbol{Q}_i \boldsymbol{K}_j^T \tag{10-34}$$

$$M_j \leftarrow \max(M_{j-1}, S_{ij}) \tag{10-35}$$

$$L_j \leftarrow L_{j-1} \mathrm{e}^{M_{j-1} - M_j} + \mathrm{e}^{S_{ij} - M_j} \tag{10-36}$$

第二个循环（$j = 1, 2, \cdots, N$）：用最终的 M_N、L_N 归一化。

$$P_{ij} \leftarrow \frac{\mathrm{e}^{S_{ij} - M_N}}{L_N} \tag{10-37}$$

按 Online-Softmax 的增量更新，初始化 $M_0 = -\infty$、$L_0 = 0$。第一个循环只累计 M_j、L_j；第二个循环用最终的 M_N、L_N 把每个分数归一化成概率。

此处仍是两次遍历：第一次求全局量，第二次才归一化。把两次合成一次，正是下一小节要做的事。

注意力输出

$$\boldsymbol{O}_i = \sum_{j=1}^{N} P_{ij}\boldsymbol{V}_j, \qquad \boldsymbol{O}_i \in \mathbb{R}^{1\times d} \tag{10-38}$$

这是标准形式：先把 N 个 P_{ij} 全部算出，再对 $\boldsymbol{V}_j$ 加权求和。

$$\boldsymbol{O}_i^{\text{old}} = \sum_{j=1}^{N-1} P_{ij}\boldsymbol{V}_j = \sum_{j=1}^{N-1} \frac{\mathrm{e}^{S_{ij}-M_{N-1}}}{L_{N-1}}\boldsymbol{V}_j \tag{10-39}$$

为推导增量形式，先写出 $N-1$ 时刻的局部注意力输出 $\boldsymbol{O}_i^{\text{old}} \in \mathbb{R}^{1\times d}$——它已经用前 $N-1$ 个词元的全局量归一化过。

若新增第 N 个键值对，M 与 L 都会变，$\boldsymbol{O}_i^{\text{old}}$ 里旧的归一化就“过时”了。需要把这份旧输出按新旧基准之比重新缩放。

依据 《大语言模型：训练、精调与对齐》第 10.2.2 节（式 (10-38)、(10-39)）

输出增量的推导

$$\boldsymbol{O}_i^{\text{new}} = \sum_{j=1}^{N} P_{ij}\boldsymbol{V}_j \tag{10-40}$$

$$= \left(\sum_{j=1}^{N-1} P_{ij}\boldsymbol{V}_j\right) + P_{iN}\boldsymbol{V}_N \tag{10-41}$$

$$= \left(\sum_{j=1}^{N-1} \frac{\mathrm{e}^{S_{ij}-M_N}}{L_N}\boldsymbol{V}_j\right) + \frac{\mathrm{e}^{S_{iN}-M_N}}{L_N}\boldsymbol{V}_N \tag{10-42}$$

$$= \left(\sum_{j=1}^{N-1} \frac{\mathrm{e}^{S_{ij}-M_{N-1}}}{L_{N-1}}\mathrm{e}^{M_{N-1}-M_N}\frac{L_{N-1}}{L_N}\boldsymbol{V}_j\right) + \frac{\mathrm{e}^{S_{iN}-M_N}}{L_N}\boldsymbol{V}_N \tag{10-43}$$

$$= \left(\sum_{j=1}^{N-1} \frac{\mathrm{e}^{S_{ij}-M_{N-1}}}{L_{N-1}}\boldsymbol{V}_j\right)\left(\mathrm{e}^{M_{N-1}-M_N}\frac{L_{N-1}}{L_N}\right) + \frac{\mathrm{e}^{S_{iN}-M_N}}{L_N}\boldsymbol{V}_N \tag{10-44}$$

$$\boldsymbol{O}_i^{\text{new}} = \boldsymbol{O}_i^{\text{old}}\left(\mathrm{e}^{M_{N-1}-M_N}\frac{L_{N-1}}{L_N}\right) + \frac{\mathrm{e}^{S_{iN}-M_N}}{L_N}\boldsymbol{V}_N \tag{10-45}$$

式 (10-41) 把第 N 项拆出求和；式 (10-42) 代入 $P_{ij}=\mathrm{e}^{S_{ij}-M_N}/L_N$，此时旧项用的是新基准；式 (10-43) 用 $\mathrm{e}^{S_{ij}-M_N}=\mathrm{e}^{S_{ij}-M_{N-1}}\mathrm{e}^{M_{N-1}-M_N}$、$\frac{1}{L_N}=\frac{1}{L_{N-1}}\cdot\frac{L_{N-1}}{L_N}$ 把旧项换回旧基准；式 (10-44) 提出与 j 无关的因子；式 (10-45) 代回式 (10-39)。

$M_{N-1}=\max(\boldsymbol{S}_{i,1:N-1})$：新块的贡献直接相加，旧块乘以由两个标量 M, L 构成的修正因子，维护 $\boldsymbol{O}$ 的代价因而很小。

输出增量的依赖关系

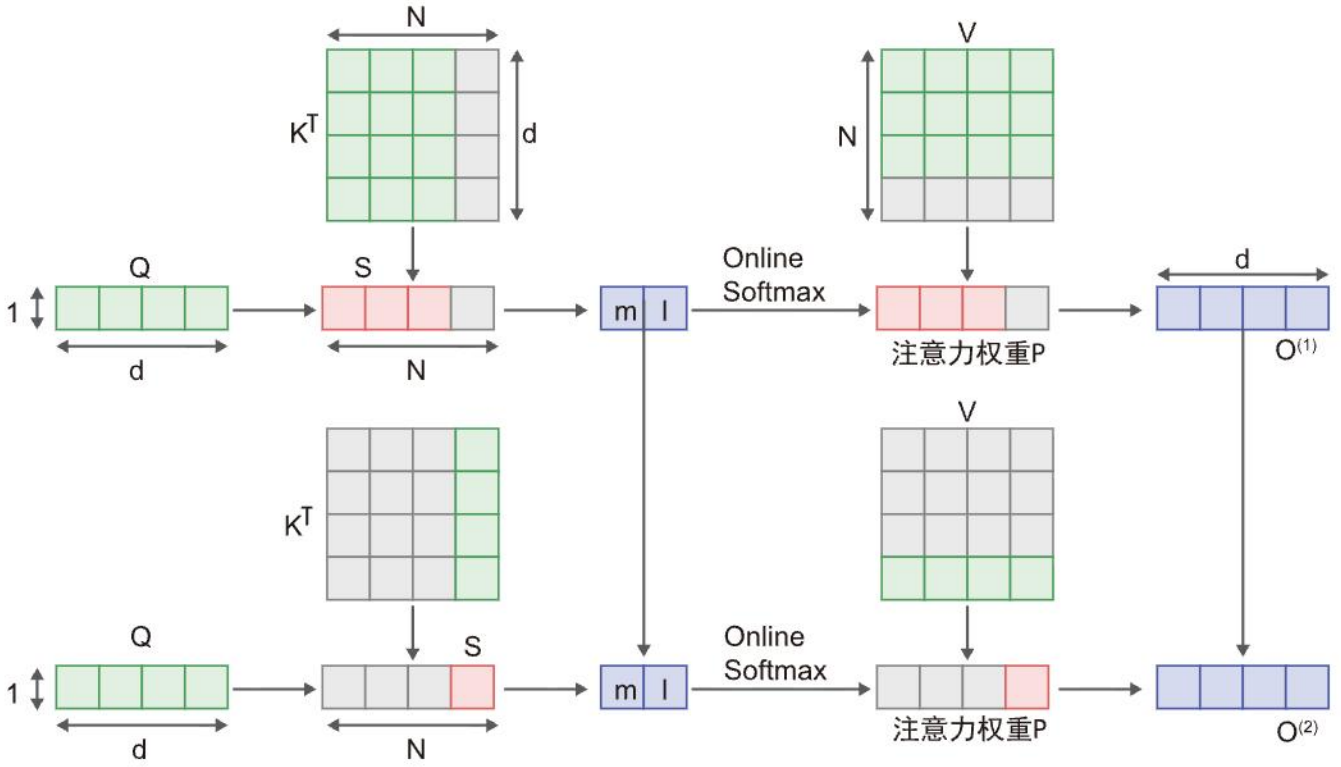


图 10.8　注意力输出随 KV 对增量更新

- 新增 $\boldsymbol{K}_j, \boldsymbol{V}_j$ 对时，$\boldsymbol{O}^{\text{new}}$ 部分依赖历史输出 $\boldsymbol{O}^{\text{old}}$。
- 依赖方式是"缩放后相加"：旧输出乘 $\mathrm{e}^{M_{N-1}-M_N}\dfrac{L_{N-1}}{L_N}$。
- 因此不必保留全部 P_{ij}，只需维护 M、L、$\boldsymbol{O}$ 三个状态。

关键　式 (10-45) 把"存储整个概率矩阵"换成了"维护一个 d 维输出向量加两个标量"。这就是 Flash Attention 省显存的根本原因。

图源　《大语言模型：训练、精调与对齐》图 10.8；式 (10-45)

单次遍历的注意力计算

对 $j = 1, 2, \cdots, N$ 依次更新三个量（式 (10-46)–(10-48)），其中最后一行给出输出。

$$S_{ij} \leftarrow \boldsymbol{Q}_i \boldsymbol{K}_j^T \tag{10-46}$$

$$M_j \leftarrow \max(M_{j-1}, S_{ij}) \tag{10-47}$$

$$L_j \leftarrow L_{j-1} \mathrm{e}^{M_{j-1} - M_j} + \mathrm{e}^{S_{ij} - M_j} \tag{10-48}$$

$$\boldsymbol{O}_i^{(j)} \leftarrow \boldsymbol{O}_i^{(j-1)} \left(\mathrm{e}^{M_{j-1} - M_j} \frac{L_{j-1}}{L_j} \right) + \frac{\mathrm{e}^{S_{ij} - M_j}}{L_j} \boldsymbol{V}_j \tag{10-49}$$

初始化 $M_0 = -\infty$、$L_0 = 0$、$\boldsymbol{O}_i^{(0)} = (\boldsymbol{0})_{1 \times d}$。每一步同时更新 M_j、L_j 与 $\boldsymbol{O}_i^{(j)}$，循环结束时 $\boldsymbol{O}_i^{(N)}$ 即为最终输出。

与式 (10-34)–(10-37) 相比

第二个循环被消掉：不再需要先算完全部 M_N, L_N 再回头归一化。

代价

旧输出要多乘一次修正因子，但省下了一整轮 $N \times N$ 的读写。

依据 《大语言模型：训练、精调与对齐》第 10.2.2 节（式 (10-46)–(10-49)）

一次循环的注意力算法

算法 17 一次循环的注意力计算

输入： $\boldsymbol{Q}_i \in \mathbb{R}^{1\times d}, \boldsymbol{K}, \boldsymbol{V} \in \mathbb{R}^{N\times d}$
输出： $\boldsymbol{O}_i \in \mathbb{R}^{1\times d}$

1: 初始化 $M \leftarrow -\infty$, $L \leftarrow 0$, $\boldsymbol{O}_i \leftarrow (\boldsymbol{0})_{1\times d}$
2: **for** $j = 1, 2, \cdots, N$ **do**
3: $\quad S_{ij} \leftarrow \boldsymbol{Q}_i \boldsymbol{K}_j^T$,
4: $\quad M^{\text{new}} \leftarrow \max(M, S_{ij})$
5: $\quad L^{\text{new}} \leftarrow L\mathrm{e}^{M-M^{\text{new}}} + \mathrm{e}^{S_{ij}-M^{\text{new}}}$
6: $\quad \boldsymbol{O}_i \leftarrow \boldsymbol{O}_i \mathrm{e}^{M-M^{\text{new}}} \frac{L}{L^{\text{new}}} + \frac{\mathrm{e}^{S_{ij}-M^{\text{new}}}}{L^{\text{new}}} \boldsymbol{V}_j$
7: $\quad M \leftarrow M^{\text{new}}, L \leftarrow L^{\text{new}}$
8: **end for**

算法 17　一次循环的注意力计算

- 第 3–5 步分别对应式 (10-47)、(10-48)、(10-49)。
- 循环体内只出现 $1 \times d$ 的输出、标量 M, L 和 $1 \times d$ 的分数行，没有 $N \times N$ 矩阵。
- 复杂度仍是 $O(N^2 d)$，但访存量从 $O(N^2)$ 降到 $O(Nd)$ 级别。

第一个块的计算

把上面的逐列遍历换成按块遍历。先算第一个数据块的局部量。

$$M_1 = M_N = \max(\boldsymbol{S}_{i,1:N}) \tag{10-50}$$

$$L_1 = L_N = \sum_{j=1}^{N} \mathrm{e}^{S_{ij} - M_1} \tag{10-51}$$

M_1 是第一块（此处即全部 N 列）的局部最大值，L_1 是以它为基准的局部指数和。块大小为 B_c，共 $T_c = N/B_c$ 块。

$$M_2 = \max(\boldsymbol{S}_{i,N+1:2N}) \tag{10-52}$$

$$\tilde{M} = \max(M_1, M_2) \tag{10-53}$$

$$L_2 = \sum_{j=N+1}^{2N} \mathrm{e}^{S_{ij} - \tilde{M}} \tag{10-54}$$

第二块的局部最大值 M_2 与全局最大值 $\tilde{M}$。注意 L_2 已经用全局基准 $\tilde{M}$ 计算，而 L_1 用的是 M_1——两者基准不同，仍需换算。

两块合并的全局指数和

$$\tilde{L} = \sum_{j=1}^{2N} \mathrm{e}^{S_{ij} - \tilde{M}} \tag{10-55}$$

$$= \left(\sum_{j=1}^{N} \mathrm{e}^{S_{ij} - \tilde{M}} \right) + \left(\sum_{j=N+1}^{2N} \mathrm{e}^{S_{ij} - \tilde{M}} \right) \tag{10-56}$$

$$= \left(\sum_{j=1}^{N} \mathrm{e}^{S_{ij} - M_1} \right) \mathrm{e}^{M_1 - \tilde{M}} + L_2 \tag{10-57}$$

$$= L_1 \mathrm{e}^{M_1 - \tilde{M}} + L_2 \tag{10-58}$$

式 (10-56) 按块拆开；式 (10-57) 把第一块换到全局基准，第二块已由式 (10-54) 用 $\tilde{M}$ 计算，故直接写作 L_2；式 (10-58) 代回 L_1 的定义。与式 (10-24) 完全同构。

M_2 之所以没有出现在式 (10-57)，是因为 L_2 的定义式 (10-54) 已经用了 $\tilde{M}$ 而非 M_2。这正是分块注意力中“每块都用当前全局最大值归一化”的写法。

两块的注意力输出

$$\boldsymbol{O}_i^{(1)} = \sum_{j=1}^{N} \frac{\mathrm{e}^{S_{ij}-M_1}}{L_1}\boldsymbol{V}_j$$

$$\boldsymbol{O}_i^{(2)} = \sum_{j=N+1}^{2N} \frac{\mathrm{e}^{S_{ij}-\tilde{M}}}{\tilde{L}}\boldsymbol{V}_j$$

$\boldsymbol{O}_i^{(1)}$ 用第一块的局部量 M_1, L_1；$\boldsymbol{O}_i^{(2)}$ 的权重基于全局 $\tilde{M}, \tilde{L}$。

- 两块输出用了不同的归一化基准，不能直接相加。
- 要合并，必须把用到局部基准的那一块换算到全局基准。
- 换算只需乘一个由 M_1, M_2, L_1, L_2 构成的标量因子。

依据 《大语言模型：训练、精调与对齐》第 10.2.2 节（分块注意力输出定义）

块输出的合并

$$\tilde{\boldsymbol{O}}_i = \sum_{j=1}^{2N} P_{ij}\boldsymbol{V}_j \tag{10-59}$$

$$= \left(\sum_{j=1}^{N} P_{ij}\boldsymbol{V}_j\right) + \left(\sum_{j=N+1}^{2N} P_{ij}\boldsymbol{V}_j\right) \tag{10-60}$$

$$= \left(\sum_{j=1}^{N} \frac{\mathrm{e}^{S_{ij}-\tilde{M}}}{\tilde{L}}\boldsymbol{V}_j\right) + \left(\sum_{j=N+1}^{2N} \frac{\mathrm{e}^{S_{ij}-\tilde{M}}}{\tilde{L}}\boldsymbol{V}_j\right) \tag{10-61}$$

$$= \left(\sum_{j=1}^{N} \frac{\mathrm{e}^{S_{ij}-M_1}}{L_1}\mathrm{e}^{M_1-\tilde{M}}\frac{L_1}{\tilde{L}}\boldsymbol{V}_j\right) + \boldsymbol{O}_i^{(2)} \tag{10-62}$$

$$= \left(\sum_{j=1}^{N} \frac{\mathrm{e}^{S_{ij}-M_1}}{L_1}\boldsymbol{V}_j\right)\left(\mathrm{e}^{M_1-\tilde{M}}\frac{L_1}{\tilde{L}}\right) + \boldsymbol{O}_i^{(2)} \tag{10-63}$$

$$= \boldsymbol{O}_i^{(1)}\mathrm{e}^{M_1-\tilde{M}}\frac{L_1}{\tilde{L}} + \boldsymbol{O}_i^{(2)} \tag{10-64}$$

式 (10-60) 按块拆开输出求和；式 (10-61) 代入全局归一化后的 P_{ij}，第二块括号内正是 $\boldsymbol{O}_i^{(2)}$；式 (10-62) 把第一块换回局部基准以便复用 $\boldsymbol{O}_i^{(1)}$，同时补上 $\mathrm{e}^{M_1-\tilde{M}}\frac{L_1}{\tilde{L}}$ 的修正；式 (10-63) 提出公共因子；式 (10-64) 代回 $\boldsymbol{O}_i^{(1)}$ 的定义。

新块的输出可以直接相加，旧块只需乘一个由两个标量构成的修正因子——这就是分块注意力可以“边算边丢”的基础。

依据 《大语言模型：训练、精调与对齐》第 10.2.2 节（式 (10-59)–(10-64)）

块注意力输出更新

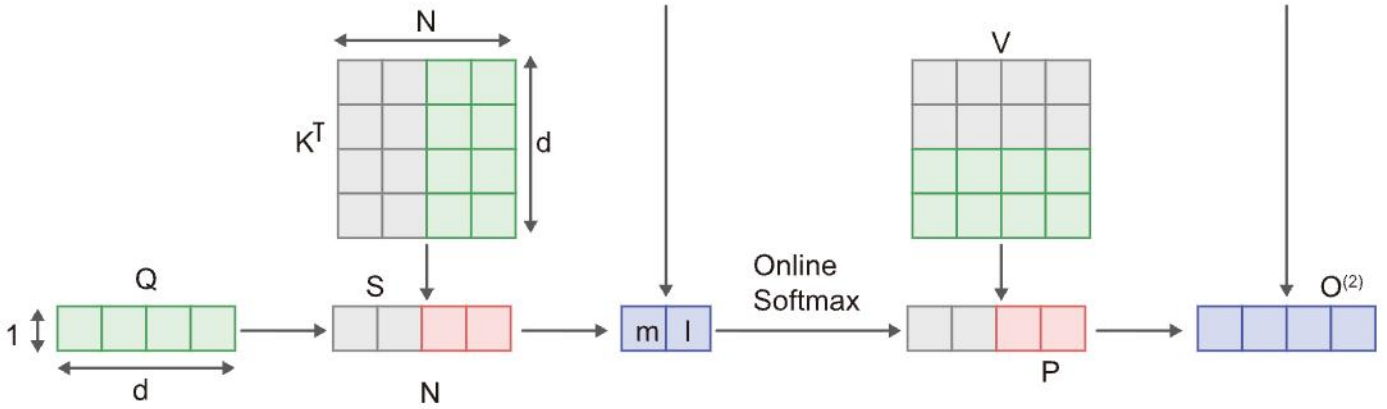


图 10.9　块注意力输出更新

- $\boldsymbol{K}, \boldsymbol{V}$ 按行切成 T_c 块，每块 $\in \mathbb{R}^{B_c \times d}$。
- 初始 $L^{\text{old}} \leftarrow 0$、$M^{\text{old}} \leftarrow -\infty$、$\boldsymbol{O}_i^{\text{old}} \leftarrow (\boldsymbol{0})_{1\times d}$。

$$M \leftarrow \max(\boldsymbol{S}_{ij}), \qquad M^{\text{new}} \leftarrow \max(M^{\text{old}}, M)$$

$$L \leftarrow \text{sum}(\boldsymbol{P}_{ij}), \qquad L^{\text{new}} \leftarrow L^{\text{old}} \mathrm{e}^{M^{\text{old}} - M^{\text{new}}} + L$$

$$\boldsymbol{O}_i^{\text{new}} \leftarrow \boldsymbol{O}_i^{\text{old}} \left(\mathrm{e}^{M^{\text{old}} - M^{\text{new}}} \frac{L^{\text{old}}}{L^{\text{new}}} \right) + \boldsymbol{O}_{ij}$$

三行依次对应式 (10-53)、(10-58)、(10-64)：先算局部量，再与旧状态合并。

图源 《大语言模型：训练、精调与对齐》图 10.9；依据该节增量更新公式

批量计算的双层循环

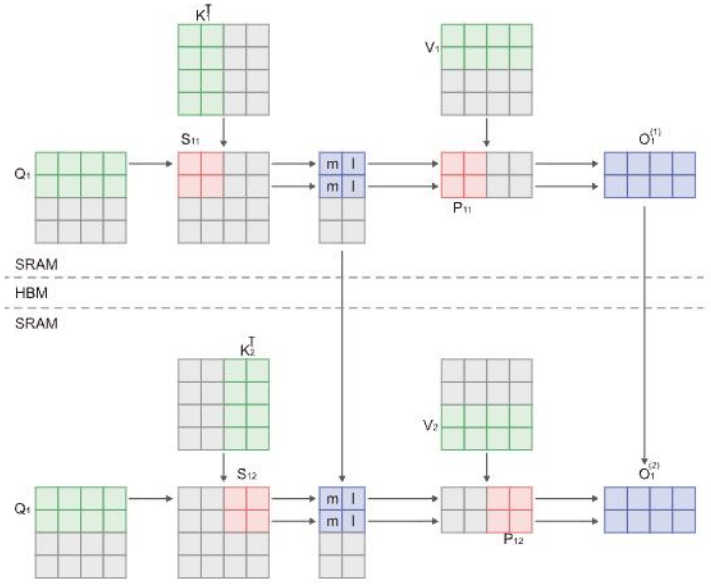


图 10.10　批量 Flash Attention 计算流程图

把 $\boldsymbol{Q} \in \mathbb{R}^{N \times d}$ 按行切成 T_r 块：$\boldsymbol{Q}_1, \cdots, \boldsymbol{Q}_{T_r} \in \mathbb{R}^{B_r \times d}$，$N/B_r = T_r$。

- 外循环遍历 $\boldsymbol{K}, \boldsymbol{V}$ 块，内循环遍历 $\boldsymbol{Q}$ 块。
- 每次计算后把更新的中间结果从 SRAM 写回 HBM。
- 处理同一查询的下一个块时，再从 HBM 加载到 SRAM。

$$\boldsymbol{S}_{ij} \leftarrow \boldsymbol{Q}_i \boldsymbol{K}_j^T, \quad \boldsymbol{S}_{ij} \in \mathbb{R}^{B_r \times B_c}; \qquad \boldsymbol{M}_{ij} \leftarrow \mathrm{rowmax}(\boldsymbol{S}_{ij}), \quad \boldsymbol{M}_{ij} \in \mathbb{R}^{B_r \times 1}$$

$$\boldsymbol{L}_{ij} \leftarrow \mathrm{rowsum}(\boldsymbol{P}_{ij}), \quad \boldsymbol{L}_{ij} \in \mathbb{R}^{B_r \times 1}$$

批量版本把标量 M、L 换成向量：每行一个最大值、一个指数和，逐块更新。

图源 《大语言模型：训练、精调与对齐》图 10.10；依据该节批量推导

批量更新式

$$\boldsymbol{S}_{ij} \leftarrow \boldsymbol{Q}_i \boldsymbol{K}_j^T, \quad \boldsymbol{S}_{ij} \in \mathbb{R}^{B_r \times B_c}; \qquad \boldsymbol{Q}_i \in \mathbb{R}^{B_r \times d}, \ \boldsymbol{K}_j^T \in \mathbb{R}^{d \times B_c} \tag{10-65}$$

$$\boldsymbol{M}_{ij} \leftarrow \mathrm{rowmax}(\boldsymbol{S}_{ij}), \quad \boldsymbol{M}_{ij} \in \mathbb{R}^{B_r \times 1} \tag{10-66}$$

$$\boldsymbol{M}_i^{\mathrm{new}} \leftarrow \mathrm{rowmax}(\boldsymbol{M}_i^{\mathrm{old}}, \boldsymbol{M}_{ij}), \quad \boldsymbol{M}_i^{\mathrm{new}} \in \mathbb{R}^{B_r \times 1} \tag{10-67}$$

$$\boldsymbol{P}_{ij} \leftarrow \mathrm{e}^{\boldsymbol{S}_{ij} - \boldsymbol{M}_i^{\mathrm{new}}}, \quad \boldsymbol{P}_{ij} \in \mathbb{R}^{B_r \times B_c} \tag{10-68}$$

$$\boldsymbol{L}_{ij} \leftarrow \mathrm{rowsum}(\boldsymbol{P}_{ij}), \quad \boldsymbol{L}_{ij} \in \mathbb{R}^{B_r \times 1} \tag{10-69}$$

$$\boldsymbol{L}_i^{\mathrm{new}} \leftarrow \mathrm{diag}(\boldsymbol{L}_i^{\mathrm{old}}) \mathrm{e}^{\boldsymbol{M}_i^{\mathrm{old}} - \boldsymbol{M}_i^{\mathrm{new}}} + \boldsymbol{L}_{ij}, \quad \boldsymbol{L}_i^{\mathrm{new}} \in \mathbb{R}^{B_r \times 1} \tag{10-70}$$

$$\boldsymbol{O}_{ij} \leftarrow \mathrm{diag}(\boldsymbol{L}_i^{\mathrm{new}})^{-1} \boldsymbol{P}_{ij} \boldsymbol{V}_j, \quad \boldsymbol{O}_{ij} \in \mathbb{R}^{B_r \times d}, \ \boldsymbol{V}_j \in \mathbb{R}^{B_c \times d} \tag{10-71}$$

$$\boldsymbol{O}_i^{\mathrm{new}} \leftarrow \mathrm{diag}(\boldsymbol{L}_i^{\mathrm{new}})^{-1} \boldsymbol{O}_i^{\mathrm{old}} \left(\mathrm{e}^{\boldsymbol{M}_i^{\mathrm{old}} - \boldsymbol{M}_i^{\mathrm{new}}} \boldsymbol{L}_i^{\mathrm{old}} \right) + \boldsymbol{O}_{ij} \tag{10-72}$$

$$\boldsymbol{L}_i^{\mathrm{old}} \leftarrow \boldsymbol{L}_i^{\mathrm{new}}, \quad \boldsymbol{M}_i^{\mathrm{old}} \leftarrow \boldsymbol{M}_i^{\mathrm{new}}, \quad \boldsymbol{O}_i^{\mathrm{old}} \leftarrow \boldsymbol{O}_i^{\mathrm{new}} \tag{10-73}$$

初始化 $\boldsymbol{M}_i^{\mathrm{old}} \leftarrow (-\infty)_{B_r \times 1}$、$\boldsymbol{L}_i^{\mathrm{old}} \leftarrow (\boldsymbol{0})_{B_r \times 1}$。外循环 $j = 1 \cdots T_c$，内循环 $i = 1 \cdots T_r$；$\mathrm{diag}(\cdot)$ 把每行的标量乘到对应行上。

最后两行把“归一化”写成了对角矩阵乘法，便于在 SRAM 内按行并行；这也是后面 V2 要消除的非矩阵乘操作来源。

SRAM 与 HBM 的数据流向

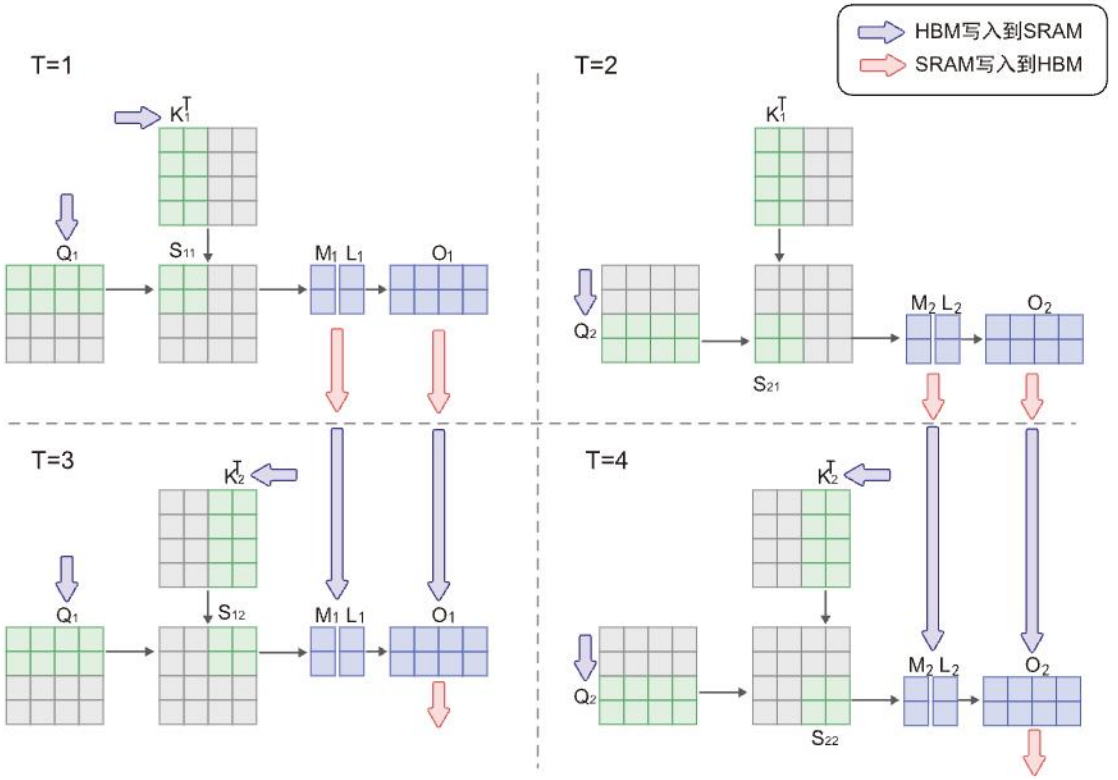


图 10.11　Flash Attention 前向计算过程数据流向图

- 外循环遍历 $\boldsymbol{K}_j\boldsymbol{V}_j$ 块，内循环遍历 $\boldsymbol{Q}_i$ 块。
- 每次算完注意力分数后，把 $\boldsymbol{M}_i$、$\boldsymbol{L}_i$、$\boldsymbol{O}_i$ 从 SRAM 写回 HBM。
- 处理相同查询的下一个块时，再从 HBM 加载回 SRAM。
- $\boldsymbol{M}_i$ 与 $\boldsymbol{L}_i$ 必须保存——反向传播要用它们重算 $\boldsymbol{P}$。

图源 《大语言模型：训练、精调与对齐》图 10.11

Flash Attention 前向算法

算法 18 Flash Attention 前向计算

输入： $\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V} \in \mathbb{R}^{N \times d}$
输出： $\boldsymbol{O} \in \mathbb{R}^{N \times D}$, $\boldsymbol{M}, \boldsymbol{L} \in \mathbb{R}^{N \times 1}$

设定块大小为 B_r, B_c，计算切分块数 $T_c = \lceil N/B_c \rceil$, $T_r = \lceil N/B_r \rceil$
将 $\boldsymbol{Q}$ 按 $B_r \times d$ 切分为 $\{\boldsymbol{Q}_1, \boldsymbol{Q}_2, \cdots, \boldsymbol{Q}_{T_r}\}$
将 $\boldsymbol{K}, \boldsymbol{V}$ 按 $B_c \times d$ 切分为 $\{\boldsymbol{K}_1, \boldsymbol{K}_2 \cdots, \boldsymbol{K}_{T_c}\}$ 和 $\{\boldsymbol{V}_1, \boldsymbol{V}_2, \cdots, \boldsymbol{V}_{T_c}\}$
初始化 $\boldsymbol{M} \leftarrow (-\infty)_{N \times 1}$, $\boldsymbol{L} \leftarrow (\boldsymbol{0})_{N \times 1}$, $\boldsymbol{O} \leftarrow (\boldsymbol{0})_{N \times D}$（存储在 HBM）
将 $\boldsymbol{M}, \boldsymbol{L}$ 按 B_r 切分为 $\{\boldsymbol{M}_1, \boldsymbol{M}_2 \cdots, \boldsymbol{M}_{T_r}\}$ 和 $\{\boldsymbol{L}_1, \boldsymbol{L}_2 \cdots, \boldsymbol{L}_{T_r}\}$, $\boldsymbol{M}_i, \boldsymbol{L}_i \in \mathbb{R}^{B_r \times 1}$
将 $\boldsymbol{O}$ 按 $B_r \times d$ 切分为 $\{\boldsymbol{O}_1, \boldsymbol{O}_2 \cdots, \boldsymbol{O}_{T_r}\}$
for $j = 1$ **to** T_c **do**
    从 HBM 加载 $\boldsymbol{K}_j, \boldsymbol{V}_j$ 到 SRAM
    for $i = 1$ **to** T_r **do**
        从 HBM 加载 $\boldsymbol{Q}_i, \boldsymbol{O}_i, \boldsymbol{L}_i, \boldsymbol{M}_i$ 到 SRAM
        计算 $\boldsymbol{S}_{ij} \leftarrow \boldsymbol{Q}_i \boldsymbol{K}_j^T \in \mathbb{R}^{B_r \times B_c}$ ▷ 芯片计算
        $\boldsymbol{M}_{ij} \leftarrow \text{rowmax}(\boldsymbol{S}_{ij})$, $\boldsymbol{M}_i^{\text{new}} \leftarrow \text{rowmax}(\boldsymbol{M}_i, \boldsymbol{M}_{ij})$ ▷ 芯片计算
        $\boldsymbol{P}_{ij} \leftarrow \exp(\boldsymbol{S}_{ij} - \boldsymbol{M}_i^{\text{new}})$, $\boldsymbol{L}_{ij} \leftarrow \text{rowsum}(\boldsymbol{P}_{ij})$ ▷ 芯片计算
        $\boldsymbol{L}_i^{\text{new}} \leftarrow \text{diag}(\boldsymbol{L}_i) \exp(\boldsymbol{M}_i - \boldsymbol{M}_i^{\text{new}}) + \boldsymbol{L}_{ij}$ ▷ 芯片计算
        $\boldsymbol{O}_{ij} \leftarrow \text{diag}(\boldsymbol{L}_i^{\text{new}})^{-1} \boldsymbol{P}_{ij} \boldsymbol{V}_j \quad \in \mathbb{R}^{B_r \times d}$ ▷ 芯片计算
        更新 $\boldsymbol{O}_i \leftarrow \text{diag}(\boldsymbol{L}_i^{\text{new}})^{-1} (\boldsymbol{O}_i \exp(\boldsymbol{M}_i - \boldsymbol{M}_i^{\text{new}}) \text{diag}(\boldsymbol{L}_i)) + \boldsymbol{O}_{ij}$ ▷ 芯片计算
        将 $\boldsymbol{O}_i, \boldsymbol{L}_i \leftarrow \boldsymbol{L}_i^{\text{new}}, \boldsymbol{M}_i \leftarrow \boldsymbol{M}_i^{\text{new}}$ 写回 HBM
    end for
end for
return $\boldsymbol{O}, \boldsymbol{L}, \boldsymbol{M}$

算法 18　Flash Attention 前向计算

Flash Attention 的加速效果

3×

GPT-2、1024 长度序列预训练，相比 Hugging Face 框架

2×

Flash Attention-V2 相比 V1 的速度提升

25–40%

V1 达到的 GEMM 理论峰值占比

- Flash Attention 利用分块计算减少高速缓存交换次数，并优化反向传播的计算方法，通过 CUDA 实现。
- V2 进一步优化循环顺序与分块策略，提升内核级并行性。
- 该系列方法已开源，并集成到 Transformers 库，替换注意力层即可即插即用。
- Flash Attention 同样适用于训练阶段，不只用于推理预填充。

衔接 V2 为什么能把峰值利用率从 25–40% 提到 50–73%？答案在 10.2.4 的任务划分与循环顺序。

Flash Attention 的反向计算

给定输出梯度 $\mathbf{d}\boldsymbol{O}_i \in \mathbb{R}^{B_r \times d} = \dfrac{\partial \mathcal{L}}{\partial \boldsymbol{O}_i}$（$\mathcal{L}$ 为误差），目标是求 $\mathbf{d}\boldsymbol{Q}, \mathbf{d}\boldsymbol{K}, \mathbf{d}\boldsymbol{V} \in \mathbb{R}^{N \times d}$。

$$\mathbf{d}\boldsymbol{V}_{ij} = \frac{\partial \boldsymbol{O}_i}{\partial \boldsymbol{P}_{ij}} = \boldsymbol{P}_{ij}^T \mathbf{d}\boldsymbol{O}_i \in \mathbb{R}^{B_c \times d} \tag{10-74}$$

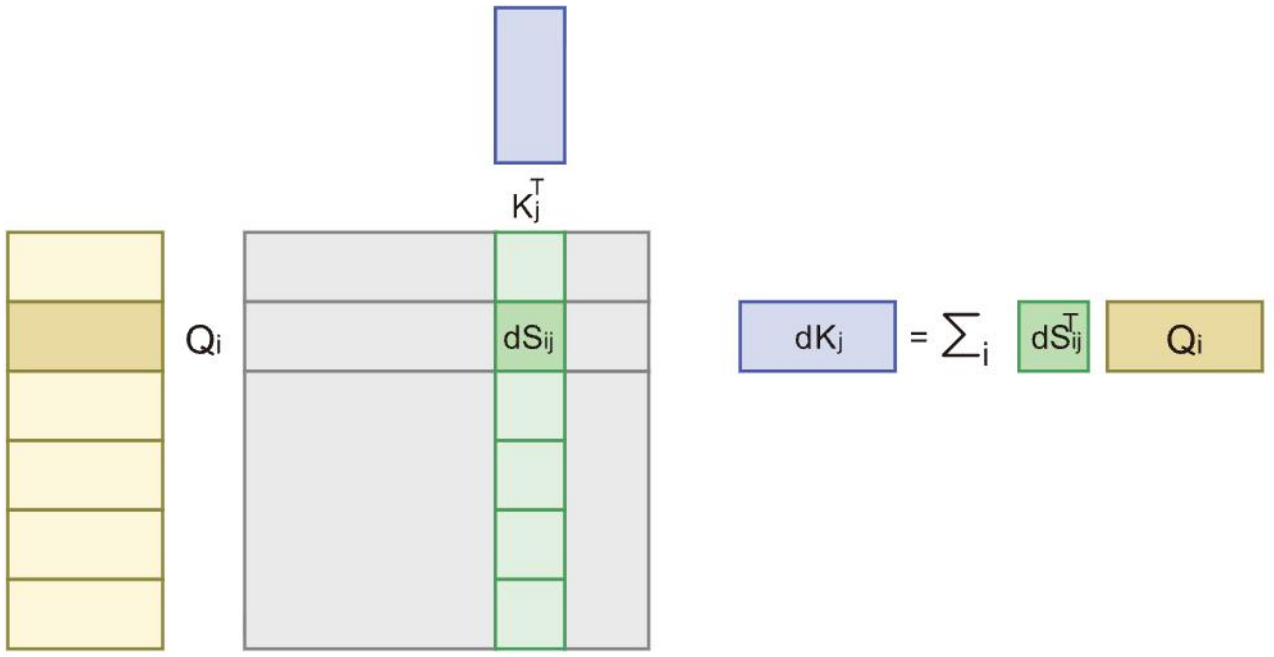
$$\mathbf{d}\boldsymbol{P}_{ij} = \frac{\partial \boldsymbol{O}_i}{\partial \boldsymbol{V}_j} = \mathbf{d}\boldsymbol{O}_i \boldsymbol{V}_j^T \in \mathbb{R}^{B_r \times B_c} \tag{10-75}$$

由 $\boldsymbol{O}_i = \boldsymbol{P}_{ij}\boldsymbol{V}_j$ 直接求导：对 $\boldsymbol{P}$ 的梯度是「输出梯度左乘 $\boldsymbol{V}^T$」，对 $\boldsymbol{V}$ 的梯度是「$\boldsymbol{P}^T$ 左乘输出梯度」。完整梯度需跨块累加：$\mathbf{d}\boldsymbol{V}_j = \sum_i^{T_r} \mathbf{d}\boldsymbol{V}_{ij}$。

$$\mathbf{d}\boldsymbol{S} = \frac{\partial \boldsymbol{O}}{\partial \boldsymbol{P}} \frac{\partial \boldsymbol{P}}{\partial \boldsymbol{S}} = \mathbf{d}\boldsymbol{P} \frac{\partial \boldsymbol{P}}{\partial \boldsymbol{S}} \tag{10-76}$$

分数矩阵的梯度经由注意力概率回传。关键是把 $\partial \boldsymbol{P}/\partial \boldsymbol{S}$ 算清楚——下一张用 Softmax 的雅可比矩阵。

$\mathrm{d}S$ 的传播



$$\mathrm{d}\boldsymbol{Q}_i = \sum_j^{T_c} \mathrm{d}\boldsymbol{Q}_{ij} = \sum_j^{T_c} \mathrm{d}\boldsymbol{S}_{ij}\boldsymbol{K}_j \tag{10-77}$$

$$\mathrm{d}\boldsymbol{K}_j = \sum_i^{T_r} \mathrm{d}\boldsymbol{K}_{ij} = \sum_i^{T_r} \mathrm{d}\boldsymbol{S}_{ij}^T\boldsymbol{Q}_i \tag{10-78}$$

由 $\boldsymbol{S}_{ij} = \boldsymbol{Q}_i\boldsymbol{K}_j^T$ 求导：$\boldsymbol{Q}$ 的梯度左乘 $\boldsymbol{K}$，$\boldsymbol{K}$ 的梯度用 $\boldsymbol{S}^T$ 左乘 $\boldsymbol{Q}$，并沿另一维度累加。

图 10.12　Flash Attention 梯度计算流程图

如图 10.12：$\mathrm{d}\boldsymbol{K}_j$ 依赖于 $\mathrm{d}\boldsymbol{S}$ 的第 j 列与 $\boldsymbol{Q}_i$；$\mathrm{d}\boldsymbol{Q}_i$ 同理，依赖 $\mathrm{d}\boldsymbol{S}$ 的第 i 行与 $\boldsymbol{K}_j$。这正是反向也能按块累加的原因。

图源 《大语言模型：训练、精调与对齐》图 10.12；式 (10-77)、(10-78)

Softmax 的雅可比矩阵

$$\frac{\partial \boldsymbol{y}}{\partial \boldsymbol{x}} = \operatorname{diag}(\boldsymbol{y}) - \boldsymbol{y}^T \boldsymbol{y} \tag{10-79}$$

$\boldsymbol{y} = \texttt{Softmax}(\boldsymbol{x})$。对角项 $\operatorname{diag}(\boldsymbol{y})$ 来自 y_i 对自身 x_i 的导数；$-\boldsymbol{y}^T\boldsymbol{y}$ 来自归一化分母对每个分量的耦合。

- 注意力分数逐行做归一化：$\boldsymbol{P}_i = \texttt{Softmax}(\boldsymbol{S}_i)$。
- 因此可以逐行套用式 (10-79)，把 $\boldsymbol{y}$ 换成 $\boldsymbol{P}_i$、$\boldsymbol{x}$ 换成 $\boldsymbol{S}_i$。
- $(\circ)$ 表示逐元素相乘。

$$\mathbf{d}\boldsymbol{S}_i = \mathbf{d}\boldsymbol{P}_i \left(\operatorname{diag}(\boldsymbol{P}_i) - \boldsymbol{P}_i^T \boldsymbol{P}_i\right) = \mathbf{d}\boldsymbol{P}_i \circ \boldsymbol{P}_i - \mathbf{d}\boldsymbol{P}_i \left(\boldsymbol{P}_i^T \boldsymbol{P}_i\right) \tag{10-80}$$

联立式 (10-76) 与式 (10-79)：第一项是雅可比的 $\operatorname{diag}(\boldsymbol{P}_i)$ 部分（逐元素），第二项是 $-\boldsymbol{P}_i^T\boldsymbol{P}_i$ 部分。

标量 D_i 的引入

$$D_i = \mathrm{d}\boldsymbol{P}_i\boldsymbol{P}_i^T = \mathrm{d}\boldsymbol{O}_i\boldsymbol{V}_j^T\boldsymbol{P}_i^T = \mathrm{d}\boldsymbol{O}_i\boldsymbol{O}_i^T = \sum_j^d \mathrm{d}\boldsymbol{O}_{ij} \circ \boldsymbol{O}_{ij} \tag{10-81}$$

$\mathrm{d}\boldsymbol{P}_i\boldsymbol{P}_i^T$ 是一个 1×1 的标量。由 $\mathrm{d}\boldsymbol{P}_i = \mathrm{d}\boldsymbol{O}_i\boldsymbol{V}_j^T$（式 (10-75)）与 $\boldsymbol{O}_i = \boldsymbol{P}_i\boldsymbol{V}$ 可把 $\boldsymbol{V}_j^T\boldsymbol{P}_i^T$ 合并成 $\boldsymbol{O}_i^T$，于是 D_i 就是 $\mathrm{d}\boldsymbol{O}_i$ 与 $\boldsymbol{O}_i$ 的逐元素内积。

$$\mathrm{d}\boldsymbol{S}_i = \mathrm{d}\boldsymbol{P}_i \circ \boldsymbol{P}_i - D_i\boldsymbol{P}_i \tag{10-82}$$

$$\mathrm{d}S_{ij} = P_{ij} \circ (\mathrm{d}P_{ij} - D_i) \tag{10-83}$$

式 (10-82) 把式 (10-80) 的第二项用 D_i 化简（矩阵乘降为标量乘）；式 (10-83) 写成逐元素形式，一行一个标量 D_i，可直接在块内计算。

结论：注意力分数的梯度可以逐元素算，因此分块反向传播不需要任何跨块的分数矩阵。

依据 《大语言模型：训练、精调与对齐》第 10.2.3 节（式 (10-81)–(10-83)）

Flash Attention 反向算法

算法 19 Flash Attention 反向计算

输入： $\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}, \boldsymbol{O}, \mathrm{d}\boldsymbol{O} \in \mathbb{R}^{N \times d}$，$\boldsymbol{L}, \boldsymbol{M} \in \mathbb{R}^{N \times 1}$

输出： $\mathrm{d}\boldsymbol{Q}, \mathrm{d}\boldsymbol{K}, \mathrm{d}\boldsymbol{V} \in \mathbb{R}^{N \times d}$

设定块大小为 B_r, B_c，计算切分块数 $T_c = \lceil N/B_c \rceil$，$T_r = \lceil N/B_r \rceil$

将 $\boldsymbol{Q}$ 按 $B_r \times d$ 行切分为 $\{\boldsymbol{Q}_1, \boldsymbol{Q}_2, \cdots, \boldsymbol{Q}_{T_r}\}$

将 $\boldsymbol{K}, \boldsymbol{V}$ 按 $B_c \times d$ 行切分为 $\{\boldsymbol{K}_1, \boldsymbol{K}_2, \cdots, \boldsymbol{K}_{T_c}\}$ 和 $\{\boldsymbol{V}_1, \boldsymbol{V}_2, \cdots, \boldsymbol{V}_{T_c}\}$

初始化 $\mathrm{d}\boldsymbol{Q} \leftarrow (\boldsymbol{0})_{N \times d}$ 分块为 $\{\mathrm{d}\boldsymbol{Q}_1, \mathrm{d}\boldsymbol{Q}_2, \cdots, \mathrm{d}\boldsymbol{Q}_{T_r}\}, \mathrm{d}\boldsymbol{Q}_i \in \mathbb{R}^{B_r \times d}$

初始化 $\mathrm{d}\boldsymbol{K} \leftarrow (\boldsymbol{0})_{N \times d}$ 分块为 $\{\mathrm{d}\boldsymbol{K}_1, \mathrm{d}\boldsymbol{K}_2, \cdots, \mathrm{d}\boldsymbol{K}_{T_c}\}, \mathrm{d}\boldsymbol{K}_j \in \mathbb{R}^{B_c \times d}$

初始化 $\mathrm{d}\boldsymbol{V} \leftarrow (\boldsymbol{0})_{N \times d}$ 分块为 $\{\mathrm{d}\boldsymbol{V}_1, \mathrm{d}\boldsymbol{V}_2, \cdots, \mathrm{d}\boldsymbol{V}_{T_c}\}, \mathrm{d}\boldsymbol{V}_j \in \mathbb{R}^{B_c \times d}$

将 $\boldsymbol{M}, \boldsymbol{L}$ 按 B_r 行切分为 $\{\boldsymbol{M}_1, \boldsymbol{M}_2, \cdots, \boldsymbol{M}_{T_r}\}$ 和 $\{\boldsymbol{L}_1, \boldsymbol{L}_2, \cdots, \boldsymbol{L}_{T_r}\}, \mathrm{d}\boldsymbol{M}_i, \mathrm{d}\boldsymbol{L}_i \in \mathbb{R}^{B_r \times 1}$

将 $\boldsymbol{O}, \mathrm{d}\boldsymbol{O}$ 按 $B_r \times d$ 行切分为 $\{\boldsymbol{O}_1, \boldsymbol{O}_2, \cdots, \boldsymbol{O}_{T_r}\}$ 和 $\{\mathrm{d}\boldsymbol{O}_1, \boldsymbol{O}_2, \cdots, \mathrm{d}\boldsymbol{O}_{T_r}\}$，$\mathrm{d}\boldsymbol{O}_i \leftarrow (\boldsymbol{0})_{B_r \times d}$

for $j = 1$ **to** T_c **do**

  从 HBM 加载 $\boldsymbol{K}_j, \boldsymbol{V}_j$ 到 SRAM

  在 SRAM 初始化 $\mathrm{d}\tilde{\boldsymbol{K}}_j \leftarrow (\boldsymbol{0})_{B_c \times d}$，$\mathrm{d}\tilde{\boldsymbol{V}}_j \leftarrow (\boldsymbol{0})_{B_c \times d}$

  for $i = 1$ **to** T_r **do**

    从 HBM 加载 $\boldsymbol{Q}_i, \mathrm{d}\boldsymbol{Q}_i, \boldsymbol{O}_i, \mathrm{d}\boldsymbol{O}_i, \boldsymbol{L}_i, \boldsymbol{M}_i$ 到 SRAM

    重计算 $\boldsymbol{S}_{ij} \leftarrow \boldsymbol{Q}_i \boldsymbol{K}_j^T \in \mathbb{R}^{B_r \times B_c}$

    重计算 $\boldsymbol{P}_{ij} \leftarrow \mathrm{diag}(\boldsymbol{L}_i)^{-1} \exp(\boldsymbol{S}_{ij} - \boldsymbol{M}_i)$

    $\mathrm{d}\tilde{\boldsymbol{V}}_j \leftarrow \mathrm{d}\tilde{\boldsymbol{V}}_j + \boldsymbol{P}_{ij}^T \mathrm{d}\boldsymbol{O}_i$

    $\mathrm{d}\boldsymbol{P}_{ij} \leftarrow \mathrm{d}\boldsymbol{O}_i \boldsymbol{V}_j^T$

    $\boldsymbol{D}_i \leftarrow \mathrm{rowsum}(\mathrm{d}\boldsymbol{O}_i \circ \boldsymbol{O}_i) \quad \in \mathbb{R}^{B_r \times 1}$

    $\mathrm{d}\boldsymbol{S}_{ij} \leftarrow \boldsymbol{P}_{ij} \circ (\mathrm{d}\boldsymbol{P}_{ij} - \boldsymbol{D}_i) \quad \in \mathbb{R}^{B_r \times B_c}$

    $\mathrm{d}\boldsymbol{Q}_i \leftarrow \mathrm{d}\boldsymbol{Q}_i + \mathrm{d}\boldsymbol{S}_{ij} \boldsymbol{K}_j$

    $\mathrm{d}\tilde{\boldsymbol{K}}_j \leftarrow \mathrm{d}\tilde{\boldsymbol{K}}_j + \mathrm{d}\boldsymbol{S}_{ij}^T \boldsymbol{Q}_i$

    将 $\mathrm{d}\boldsymbol{Q}_i$ 写入到 HBM

  end for

  将 $\mathrm{d}\boldsymbol{K}_j \leftarrow \mathrm{d}\tilde{\boldsymbol{K}}_j$，$\mathrm{d}\boldsymbol{V}_j \leftarrow \mathrm{d}\tilde{\boldsymbol{V}}_j$ 写入 HBM

end for

return $\mathrm{d}\boldsymbol{Q}, \mathrm{d}\boldsymbol{K}, \mathrm{d}\boldsymbol{V}$

算法 19　Flash Attention 反向计算

重计算策略

为降低存储占用，可采用重计算策略减少激活值存储：Softmax 算子的输出激活值可在使用后丢弃。

$$\boldsymbol{P}_{ij} = \mathrm{diag}(\boldsymbol{L}_i)^{-1} \exp(\boldsymbol{S}_{ij} - \boldsymbol{M}_i) \tag{10-84}$$

反向时用前向保存的 $\boldsymbol{M}_i$（行最大值）与 $\boldsymbol{L}_i$（行指数和）重算 $\boldsymbol{P}_{ij}$；其中 $\boldsymbol{S}_{ij} = \boldsymbol{Q}_i \boldsymbol{K}_j^T$ 也现场重算。

保存的

$\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}, \boldsymbol{O}$ 与两个 $N \times 1$ 向量 $\boldsymbol{M}, \boldsymbol{L}$。

不保存的

$\boldsymbol{S}$ 与 $\boldsymbol{P}$ 两个 $N \times N$ 矩阵，反向时按块重算。

V1 的性能

Flash Attention 的计算效率仅达到通用矩阵乘法（GEMM）操作理论峰值的 25%–40%。

瓶颈诊断

任务划分策略不合理

GPU 线程块与线程束的任务划分不当，导致线程块占用率较低，共享内存读写冗余。

改进 1

减少非矩阵乘法运算开销。

改进 2

在单头注意力中沿 K、V 的序列维度切分任务，提升线程块占用率。

改进 3

在线程块内跨线程束分配任务，降低共享内存通信开销。

2×

相比 V1 的加速比

50–73%

A100 上达到的 GEMM 理论峰值占比

225

GPT-2 训练中的 TFLOPs/s，约 72% 硬件峰值

依据 《大语言模型：训练、精调与对齐》第 10.2.4 节

V1 的访存问题

V1 采用 $\boldsymbol{K}$、$\boldsymbol{V}$ 优先切分策略：各块完成局部注意力计算后，需把中间状态 $\boldsymbol{O}_i$、$\boldsymbol{M}_i$、$\boldsymbol{L}_i$ 与 HBM 交换。

V1：K, V 外循环

每个 $\boldsymbol{Q}$ 块在每轮外循环中都要重新加载 $\boldsymbol{O}_i, \boldsymbol{M}_i, \boldsymbol{L}_i$。

线程束之间共享中间状态，必须经共享内存通信。

V2：Q 外循环

各线程束独立处理 $\boldsymbol{Q}$ 子集，无需跨线程束通信。

$\boldsymbol{K}, \boldsymbol{V}$ 在内循环被所有线程束共享，单次循环即可完成全局注意力。

> **结果** 中间状态 $\boldsymbol{M}_i$、$\boldsymbol{L}_i$、$\boldsymbol{O}_i$ 全程留在片上，不再与 HBM 来回交换。

V2 的循环顺序

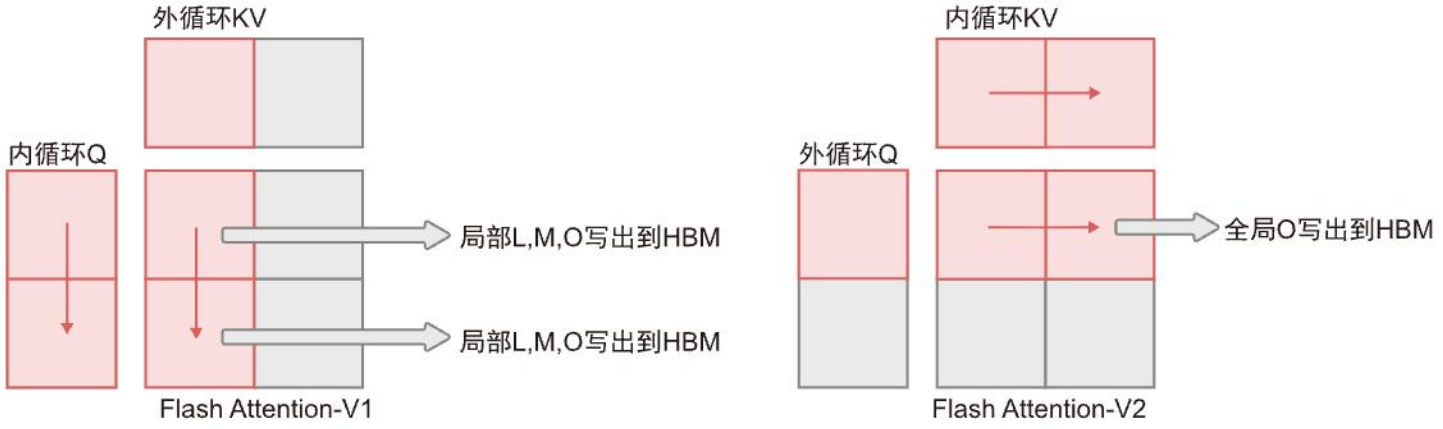


图 10.13　Flash Attention 循环顺序对比图

- $\boldsymbol{Q}$ 置于外循环，$\boldsymbol{K}, \boldsymbol{V}$ 置于内循环。
- 各线程束独立处理 $\boldsymbol{Q}$ 子集，无需跨线程束通信。
- $\boldsymbol{K}, \boldsymbol{V}$ 被所有线程束共享，沿序列维度切分后单次循环即可完成全局注意力。

$$\boldsymbol{M}_i \leftarrow \mathrm{rowmax}(\boldsymbol{S}_{ij}), \qquad \boldsymbol{L}_i \leftarrow \mathrm{rowsum}\left(\exp(\boldsymbol{S}_{ij} - \boldsymbol{M}_i)\right)$$

$\boldsymbol{O}_i$ 全程留在片上，循环结束才写回 HBM

V2 的循环体内，三个状态只在本线程束的 SRAM 中更新，循环结束时才写回一次。

图源 《大语言模型：训练、精调与对齐》图 10.13

Flash Attention V2 前向算法

算法 20 Flash Attention-V2 前向计算

输入： 矩阵 $\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V} \in \mathbb{R}^{N \times d}$, 块大小 B_c, B_r
输出： $\boldsymbol{O} \in \mathbb{R}^{N \times d}$ 和 $\boldsymbol{L}' \in \mathbb{R}^{N \times 1}$

设定块大小为 B_r, B_c，计算切分块数 $T_c = \lceil N/B_c \rceil$, $T_r = \lceil N/B_r \rceil$
将 $\boldsymbol{Q}$ 按 $B_r \times d$ 行切分为 $\{\boldsymbol{Q}_1, \boldsymbol{Q}_2, \cdots, \boldsymbol{Q}_{T_r}\}$
将 $\boldsymbol{K}, \boldsymbol{V}$ 按 $B_c \times d$ 切分为 $\{\boldsymbol{K}_1, \boldsymbol{K}_2, \cdots, \boldsymbol{K}_{T_c}\}$ 和 $\{\boldsymbol{V}_1, \boldsymbol{V}_2, \cdots, \boldsymbol{V}_{T_c}\}$
初始化 $\boldsymbol{M} \leftarrow (-\infty)_{N \times 1}$, $\boldsymbol{L} \leftarrow (\boldsymbol{0})_{N \times 1}$, $\boldsymbol{O} \leftarrow (\boldsymbol{0})_{N \times D}$（存储在 HBM）
for $i = 1$ to T_r **do**
    从 HBM 加载 $\boldsymbol{Q}_i$ 到片上 SRAM
    在片上初始化 $\boldsymbol{O}_i \leftarrow (\boldsymbol{0})_{B_r \times d}, \boldsymbol{L}_i \leftarrow (\boldsymbol{0})_{B_r \times 1}, \boldsymbol{M}_i \leftarrow (-\infty)_{B_r \times 1}$
    for $j = 1$ to T_c **do**
        从 HBM 加载 $\boldsymbol{K}_j, \boldsymbol{V}_j$ 到片上 SRAM
        在片上计算 $\boldsymbol{S}_{ij} \leftarrow \boldsymbol{Q}_i \boldsymbol{K}_j^T \in \mathbb{R}^{B_r \times B_c}$
        在片上计算 $\boldsymbol{M}_i^{\text{new}} \leftarrow \text{rowmax}(\boldsymbol{M}_i, \text{rowmax}(\boldsymbol{S}_{ij})) \in \mathbb{R}^{B_r \times 1}$
        在片上计算 $\boldsymbol{P}_{ij} \leftarrow \exp(\boldsymbol{S}_{ij} - \boldsymbol{M}_i^{\text{new}}) \in \mathbb{R}^{B_r \times B_c}$
        在片上计算 $\boldsymbol{L}_i^{\text{new}} \leftarrow \text{diag}(\boldsymbol{L}_i) \exp(\boldsymbol{M}_i - \boldsymbol{M}_i^{\text{new}}) + \text{rowsum}(\boldsymbol{P}_{ij}) \in \mathbb{R}^{B_r \times 1}$
        在片上计算 $\boldsymbol{O}_i \leftarrow \exp(\boldsymbol{M}_i - \boldsymbol{M}_i^{\text{new}}) \boldsymbol{O}_i + \boldsymbol{P}_{ij} \boldsymbol{V}_j$ ▷ 无 diag 算子
        更新 $\boldsymbol{M}_i \leftarrow \boldsymbol{M}_i^{\text{new}}, \boldsymbol{L}_i \leftarrow \boldsymbol{L}_i^{\text{new}}$
    end for
    重缩放注意力输出 $\boldsymbol{O}_i = \text{diag}(\boldsymbol{L}_i)^{-1} \boldsymbol{O}_i$
    $\boldsymbol{L}_i' = \boldsymbol{M}_i + \log(\boldsymbol{L}_i)$
    将 $\boldsymbol{O}_i$ 作为 $\boldsymbol{O}$ 的 第 i 个块写到 HBM
    将 $\boldsymbol{L}_i'$ 作为 $\boldsymbol{L}'$ 的 第 i 个块写到 HBM
end for
return 输出 $\boldsymbol{O}$ 和 $\boldsymbol{L}'$

算法 20　Flash Attention-V2 前向计算

重缩放的消除

$$\boldsymbol{M}_i = \mathrm{rowmax}(\boldsymbol{S}_{ij}) \tag{10-85}$$

$$\boldsymbol{O}_i = \mathrm{diag}(\boldsymbol{L}_i)^{-1} \sum_{j=1}^{N} \mathrm{e}^{S_{ij}-M_i} \boldsymbol{V}_j \tag{10-86}$$

$$\tilde{\boldsymbol{O}}_i^{\mathrm{old}} = \sum_{j}^{N-1} \mathrm{e}^{S_{ij}-M_i^{\mathrm{old}}} \boldsymbol{V}_j \tag{10-87}$$

$$\tilde{\boldsymbol{O}}_i^{\mathrm{new}} = \sum_{j=1}^{N} \mathrm{e}^{S_{ij}-M_i^{\mathrm{new}}} \boldsymbol{V}_j = \tilde{\boldsymbol{O}}_i^{\mathrm{old}} \mathrm{e}^{M_i^{\mathrm{old}}-M_i^{\mathrm{new}}} + \mathrm{e}^{S_{iN}-M_i^{\mathrm{new}}} \boldsymbol{V}_N \tag{10-88}$$

$$\boldsymbol{O}_i = \mathrm{diag}(\boldsymbol{L}_i^{\mathrm{new}})^{-1} \tilde{\boldsymbol{O}}_i \tag{10-89}$$

式 (10-85)、(10-86) 是传统写法：$\boldsymbol{O}_i$ 每次更新都带着 $\mathrm{diag}(\boldsymbol{L}_i)^{-1}$ 这一非矩阵乘操作。式 (10-87)、(10-88) 先不归一化，只累加带指数的输出和；把求和拆成前 $N-1$ 项与第 N 项后，前 $N-1$ 项正是 $\tilde{\boldsymbol{O}}_i^{\mathrm{old}}$ 换到新基准的结果。式 (10-89) 把归一化推迟到内循环结束，$\mathrm{diag}(\cdot)^{-1}$ 从每次迭代一次降到整轮一次。

式 (10-85)、(10-86)

每次更新都做一次非矩阵乘的归一化。

式 (10-87)、(10-88)

只做乘法与加法，与式 (10-43)–(10-45) 同构但去掉了 L 的比值。

式 (10-89)

内循环结束后归一化一次，得到全局输出。

依据 《大语言模型：训练、精调与对齐》第 10.2.4 节（式 (10-85)–(10-89)）

V2 的反向计算

反向采用 $\boldsymbol{K}, \boldsymbol{V}$ 为外循环、$\boldsymbol{Q}$ 为内循环的切分策略，相比 V1 可减少约 50% 的访存。利用前向存储的 $\boldsymbol{L}'_i$，可以精确重构 Softmax 概率。

$$\begin{aligned}\boldsymbol{P}_{ij} &= \exp\left(\boldsymbol{S}_{ij} - \boldsymbol{L}'_i\right) = \exp\left(\boldsymbol{S}_{ij} - (\boldsymbol{M}_i + \log(\boldsymbol{L}_i))\right) \\ &= \exp\left(\boldsymbol{S}_{ij} - \boldsymbol{M}_i - \log(\boldsymbol{L}_i)\right) = \operatorname{diag}\left(\exp\left(-\log(\boldsymbol{L}_i)\right)\right) \exp\left(\boldsymbol{S}_{ij} - \boldsymbol{M}_i\right) \\ &= \operatorname{diag}(\boldsymbol{L}_i)^{-1} \exp\left(\boldsymbol{S}_{ij} - \boldsymbol{M}_i\right)\end{aligned} \tag{10-90}$$

逐步代入 $\boldsymbol{L}'$ 的定义、展开对数的加减、把 $-\log(\boldsymbol{L}_i)$ 写成对角矩阵的指数，最后用 $\exp(-\log x) = 1/x$ 收拢。

$$\boldsymbol{L}' = \boldsymbol{M} + \log(\boldsymbol{L})$$

前向实际保存的是 Log-Sum-Exp 值 $\boldsymbol{L}'$：不再同时保留行最大值 $\boldsymbol{M}$ 与指数和 $\boldsymbol{L}$，两个 $N \times 1$ 向量合并为一个。它由前向直接得出，反向时可从中精确还原 $\operatorname{diag}(\boldsymbol{L}_i)^{-1} \exp(\boldsymbol{S}_{ij} - \boldsymbol{M}_i)$。

存储

两个 $N \times 1$ 向量合并为一个 $\boldsymbol{L}'$，反向的加载量减半。

计算

内循环只剩矩阵乘与逐元素乘加，非矩阵乘操作移到循环外。

依据 《大语言模型：训练、精调与对齐》第 10.2.5 节（式 (10-90) 与 $\boldsymbol{L}'$ 的定义）

Flash Attention V2 反向算法

算法 21 Flash Attention-V2 反向计算

输入: 矩阵 $\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}, \boldsymbol{O}, \mathrm{d}\boldsymbol{O} \in \mathbb{R}^{N \times d}$, $\boldsymbol{L}' \in \mathbb{R}^{N \times 1}$

输出: $\mathrm{d}\boldsymbol{Q}, \mathrm{d}\boldsymbol{K}, \mathrm{d}\boldsymbol{V}$

设定块大小为 B_r, B_c，计算切分块数 $T_c = \lceil N/B_c \rceil$, $T_r = \lceil N/B_r \rceil$

将 $\boldsymbol{Q}$ 按 $B_r \times d$ 切分为 $\{\boldsymbol{Q}_1, \boldsymbol{Q}_2, \cdots, \boldsymbol{Q}_{T_r}\}$

将 $\boldsymbol{K}, \boldsymbol{V}$ 按 $B_c \times d$ 切分为 $\{\boldsymbol{K}_1, \boldsymbol{K}_2, \cdots, \boldsymbol{K}_{T_c}\}$ 和 $\{\boldsymbol{V}_1, \boldsymbol{V}_2, \cdots, \boldsymbol{V}_{T_c}\}$

初始化 $\mathrm{d}\boldsymbol{Q} \leftarrow (\boldsymbol{0})_{N \times d}$ 分块为 $\{\mathrm{d}\boldsymbol{Q}_1, \mathrm{d}\boldsymbol{Q}_2, \cdots, \mathrm{d}\boldsymbol{Q}_{T_r}\}$

初始化 $\mathrm{d}\boldsymbol{K} \leftarrow (\boldsymbol{0})_{N \times d}$ 分块为 $\{\mathrm{d}\boldsymbol{K}_1, \mathrm{d}\boldsymbol{K}_2, \cdots, \mathrm{d}\boldsymbol{K}_{T_c}\}$

初始化 $\mathrm{d}\boldsymbol{V} \leftarrow (\boldsymbol{0})_{N \times d}$ 分块为 $\{\mathrm{d}\boldsymbol{V}_1, \mathrm{d}\boldsymbol{V}_2, \cdots, \mathrm{d}\boldsymbol{V}_{T_c}\}$

将 $\boldsymbol{L}'$ 按 B_r 切分为 $\{\boldsymbol{L}'_1, \boldsymbol{L}'_2, \cdots, \boldsymbol{L}'_{T_r}\}, \boldsymbol{L}_i \in \mathbb{R}^{B_r \times 1}$

将 $\boldsymbol{O}, \mathrm{d}\boldsymbol{O}$ 按 $B_r \times d$ 切分为 $\{\boldsymbol{O}_1, \boldsymbol{O}_2, \cdots, \boldsymbol{O}_{T_r}\}$ 和 $\{\mathrm{d}\boldsymbol{O}_1, \mathrm{d}\boldsymbol{O}_2, \cdots, \mathrm{d}\boldsymbol{O}_{T_r}\}$

计算 $\boldsymbol{D} = \mathrm{rowsum}(\mathrm{d}\boldsymbol{O} \circ \boldsymbol{O}) \in \mathbb{R}^{N \times 1}$,

将 $\boldsymbol{D}$ 按 B_r 切分为 $\{\boldsymbol{D}_1, \boldsymbol{D}_2, \cdots, \boldsymbol{D}_{T_r}\}, \boldsymbol{D}_i \in \mathbb{R}^{B_r \times 1}$

for $j = 1$ to T_c **do**

  从 HBM 加载 $\boldsymbol{K}_j, \boldsymbol{V}_j$ 到片上 SRAM

  片上 SRAM 初始化 $\mathrm{d}\boldsymbol{K}_j \leftarrow (\boldsymbol{0})_{B_c \times d}$, $\mathrm{d}\boldsymbol{V}_j \leftarrow (\boldsymbol{0})_{B_c \times d}$

  for $i \leftarrow 1$ to T_r **do**

    从 HBM 加载 $\boldsymbol{Q}_i, \boldsymbol{O}_i, \mathrm{d}\boldsymbol{O}_i, \mathrm{d}\boldsymbol{Q}_i, \boldsymbol{L}'_i, \boldsymbol{D}_i$ 到片上 SRAM。

    在片上计算: $\boldsymbol{S}_{ij} \leftarrow \boldsymbol{Q}_i \boldsymbol{K}_j^T \in \mathbb{R}^{B_r \times B_c}$

    在片上计算: $\boldsymbol{P}_{ij} \leftarrow \exp(\boldsymbol{S}_{ij} - \boldsymbol{L}'_i) \in \mathbb{R}^{B_r \times B_c}$

    在片上计算: $\mathrm{d}\boldsymbol{V}_j \leftarrow \mathrm{d}\boldsymbol{V}_j + \boldsymbol{P}_{ij}^T \mathrm{d}\boldsymbol{O}_i \in \mathbb{R}^{B_c \times d}$

    在片上计算: $\mathrm{d}\boldsymbol{P}_{ij} \leftarrow \mathrm{d}\boldsymbol{O}_i \boldsymbol{V}_j^T \in \mathbb{R}^{B_r \times B_c}$

    在片上计算: $\mathrm{d}\boldsymbol{S}_{ij} \leftarrow \boldsymbol{P}_{ij} \circ (\mathrm{d}\boldsymbol{P}_{ij} - \boldsymbol{D}_i) \in \mathbb{R}^{B_r \times B_c}$

    更新 $\mathrm{d}\boldsymbol{Q}_i \leftarrow \mathrm{d}\boldsymbol{Q}_i + \mathrm{d}\boldsymbol{S}_{ij} \boldsymbol{K}_j \in \mathbb{R}^{B_r \times d}$, 并写回到 SRAM

    在片上计算: $\mathrm{d}\boldsymbol{K}_j \leftarrow \mathrm{d}\boldsymbol{K}_j + \mathrm{d}\boldsymbol{S}_{ij}^T \boldsymbol{Q}_i$

  end for

  将 $\mathrm{d}\boldsymbol{K}_j, \mathrm{d}\boldsymbol{V}_j$ 写到 HBM

end for

return $\mathrm{d}\boldsymbol{Q}, \mathrm{d}\boldsymbol{K}, \mathrm{d}\boldsymbol{V}$

算法 21　Flash Attention-V2 反向计算

从单卡分块到上下文并行

Flash Attention-V2 显著优化了长序列场景下的注意力计算效率。环状注意力进一步把分块思想扩展到上下文并行维度。

单卡分块

V1：$\boldsymbol{K}, \boldsymbol{V}$ 外层切分

→

循环重排

V2：$\boldsymbol{Q}$ 外循环，占用率提升

→

减少非矩阵乘

$\tilde{\boldsymbol{O}}$ 累加 + $\boldsymbol{L}'$

→

跨设备

环状注意力，分布式预填充加速

- 环状注意力在分布式训练和推理预填充场景中实现注意力计算的加速。
- 分块、增量聚合、重计算这三个技巧在后续 vLLM 的分页注意力中会再次出现。

依据 《大语言模型：训练、精调与对齐》第 10.2.5 节末段

10.3

vLLM 推理服务

本节内容

10.3.1 推理服务问题
10.3.2 分页注意力机制
10.3.3 连续批处理
10.3.4 解码策略

解码是存储密集型任务：能同时解码多少请求，取决于 KV-Cache 管得多细。vLLM 把操作系统的分页机制搬进了显存。

推理服务的场景特点

大语言模型部署涉及嵌入式设备与云计算服务。云端通常要在 GPU 集群上对外提供服务，需针对 Transformer 的推理特性设计专用系统。

预填充任务

计算密集型

整条提示词一次前向，矩阵运算充分并行，决定首词元延迟。

解码任务

存储密集型

每步只处理一个词元，瓶颈在 KV-Cache 的读取与容量。

低延时 + 高吞吐

在有限资源下同时做到尽可能快的响应与尽可能大的批量。

解码批次量是关键

解码阶段串行推进，有限存储下能塞进多少请求决定吞吐。

两阶段可分离

按计算特性把预填充与解码分开部署，提高整体系统效率。

依据 《大语言模型：训练、精调与对齐》第 10.3 节

显存分配的碎片问题

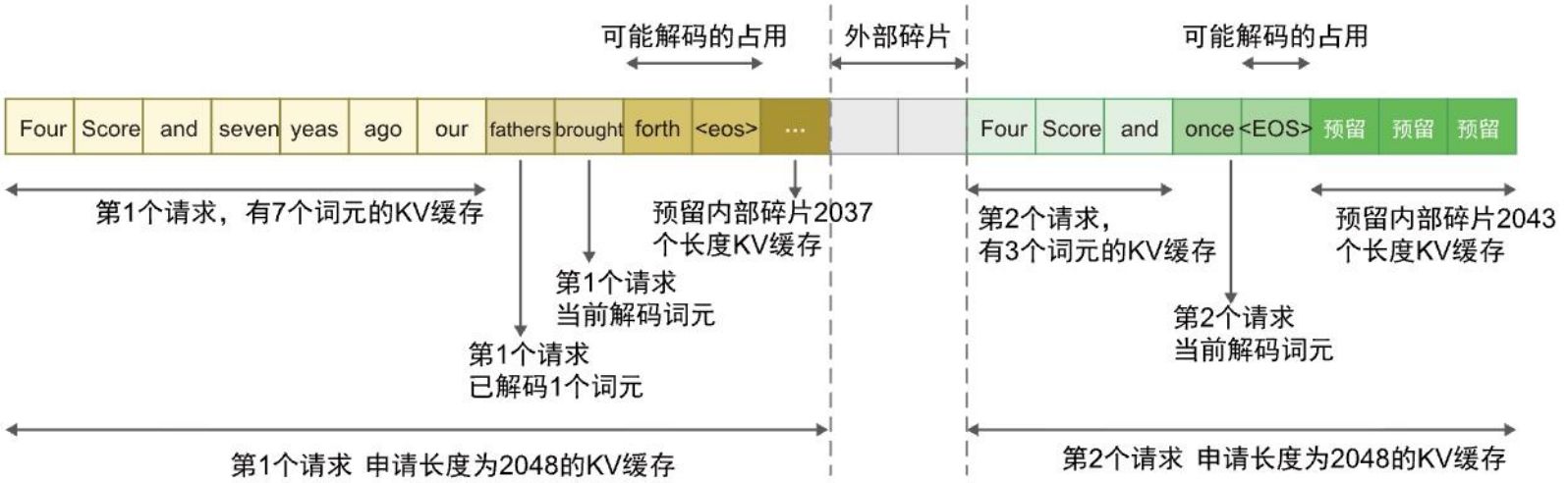


图 10.14　LLM 推理的 KV-Cache 存储碎片示例

内部碎片

为每个请求预留固定长度空间，解码提前终止后，预留部分被浪费。

外部碎片

不同请求的 KV-Cache 之间的间隙无法被其他请求使用。

两种浪费同时放大

用户提示词长度不确定、到达随机。

图 10.14 中，第一条请求含 7 个词元，额外预留 2048 词元空间，却只生成 4 个词元即终止；第二条请求同样预留固定长度，两条之间的间隙成为外部碎片。

图源 《大语言模型：训练、精调与对齐》图 10.14

键值缓存的显存占用

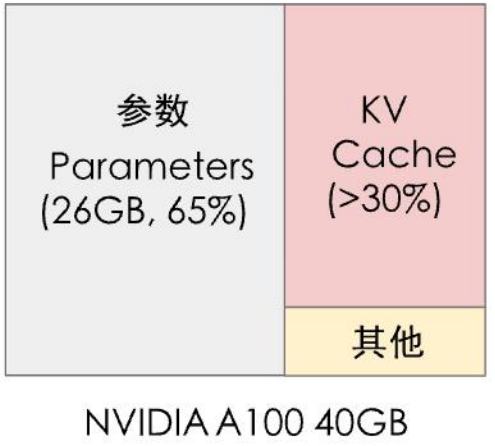


图 10.15(a)　显存占用分布

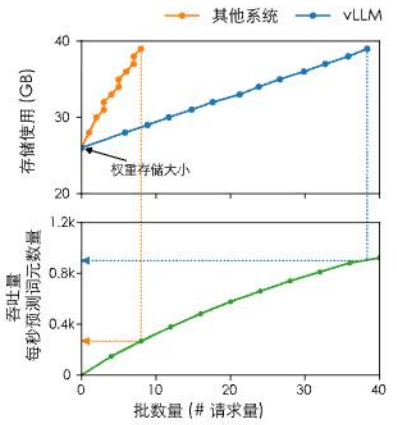


图 10.15(b)　vLLM 推理效率

图 10.15　KV-Cache 利用率和吞吐量之间的关系

- 以 A100（40GB）为例，KV-Cache 占用超过 30% 显存。
- 受限于存储墙，传统系统难以提升批量大小，进而限制推理速度。
- 高效管理 KV-Cache 可支持更大批量：vLLM 的词元生成速率较现有系统提升约 3 倍，可支撑的解码批量显著更大。

图源 《大语言模型：训练、精调与对齐》图 10.15(a)(b)

显存利用率的差距

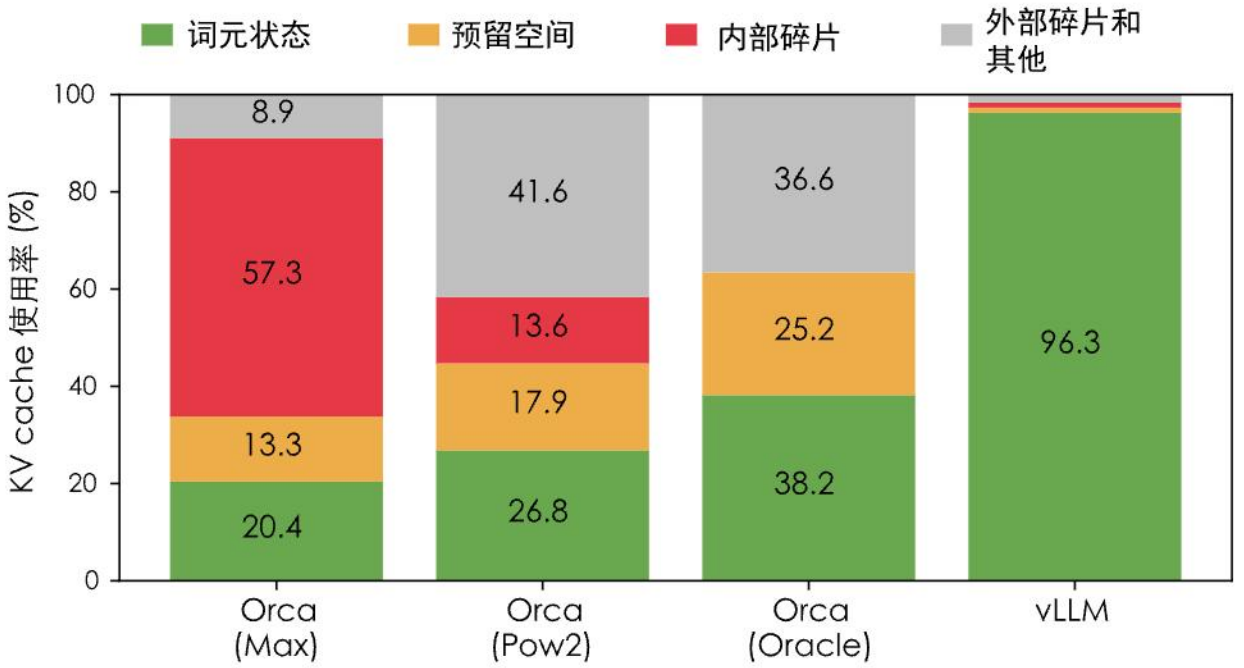


图 10.16　vLLM 系统的 KV-Cache 管理效率

38.2%

Orca 的 KV-Cache 利用率

≈100%

vLLM 的利用率，接近零浪费

- 传统管理策略在“预留”与“对齐”上浪费了大量可用空间。
- vLLM 让几乎每一个显存页都能存放有效 KV-Cache。

图源 《大语言模型：训练、精调与对齐》图 10.16

三种显存管理策略

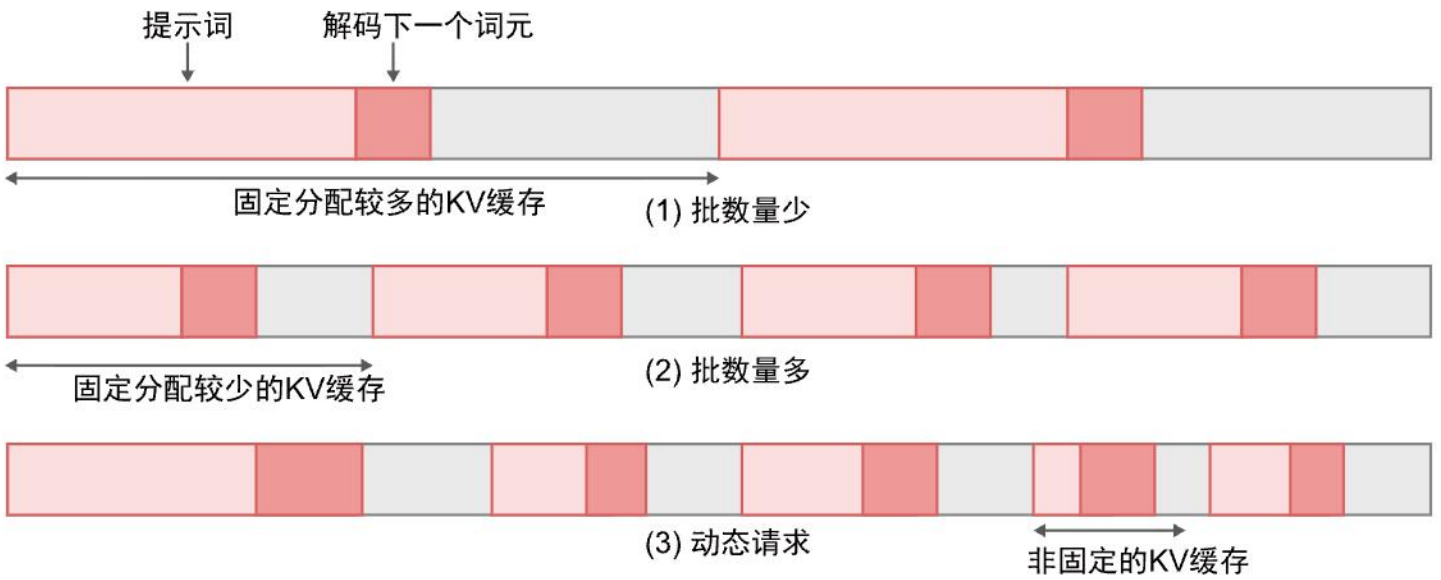


图 10.17　分块 KV-Cache 管理系统

策略	后果
(a) 固定大块	块内用不满，产生碎片
(b) 固定小块	限制单请求可处理的长度
(c) 分块管理	动态块分配，请求可跨越多个**非连续**存储块

关键差别：分块管理允许一个请求的 KV-Cache 不连续存放，因此不必为“最长可能序列”预留连续空间。

图源 《大语言模型：训练、精调与对齐》图 10.17

显存管理策略的对比

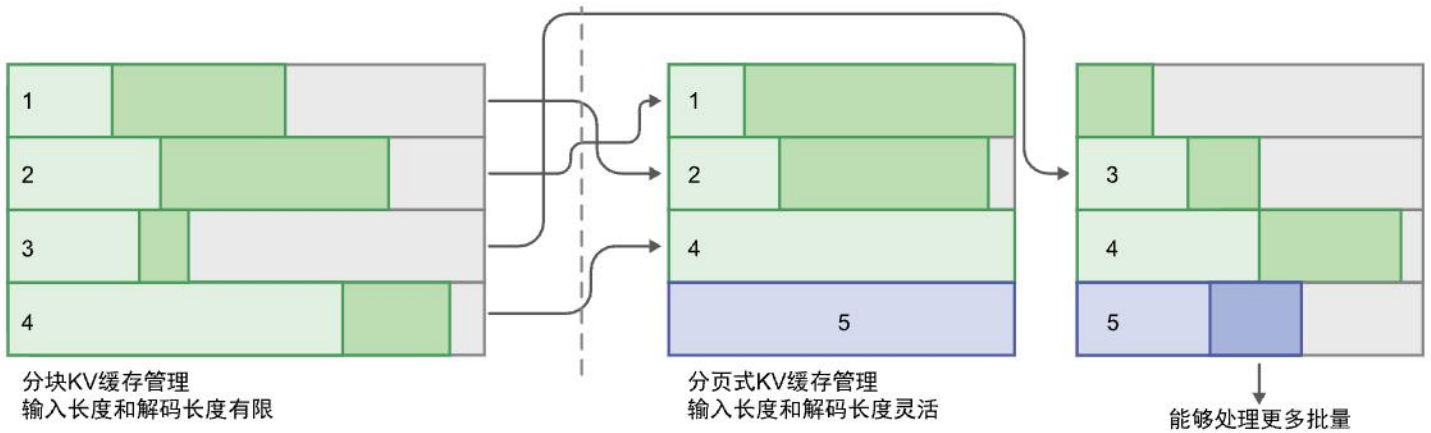


图 10.18　灵活的 KV-Cache 存储管理提高处理请求数量

- 原始 4 个存储块只支持 4 个请求。
- 分块管理后，请求 2 和 4 可以跨块存储。
- 新增的请求 5 可利用空闲块，同样存储大小下多承载一个请求。
- 块与块之间不要求连续。

图源 《大语言模型：训练、精调与对齐》图 10.18

页式显存管理

vLLM 借鉴操作系统的分页式存储管理：将物理显存划分为固定大小的页，并建立虚拟地址到物理地址的映射关系。

请求到达
序列长度未知

→

分配虚拟页
不要求物理连续

→

块表
虚拟页 → 物理页

→

填满即追加
动态分配新物理页

- 避免了为每个请求预先分配连续存储空间的需求。
- 序列长度超过单页容量时，虚拟内存管理会动态分配新的物理页。
- 同一请求的 KV-Cache 因而可以分布在非连续的物理存储空间中。

vLLM 的系统组成

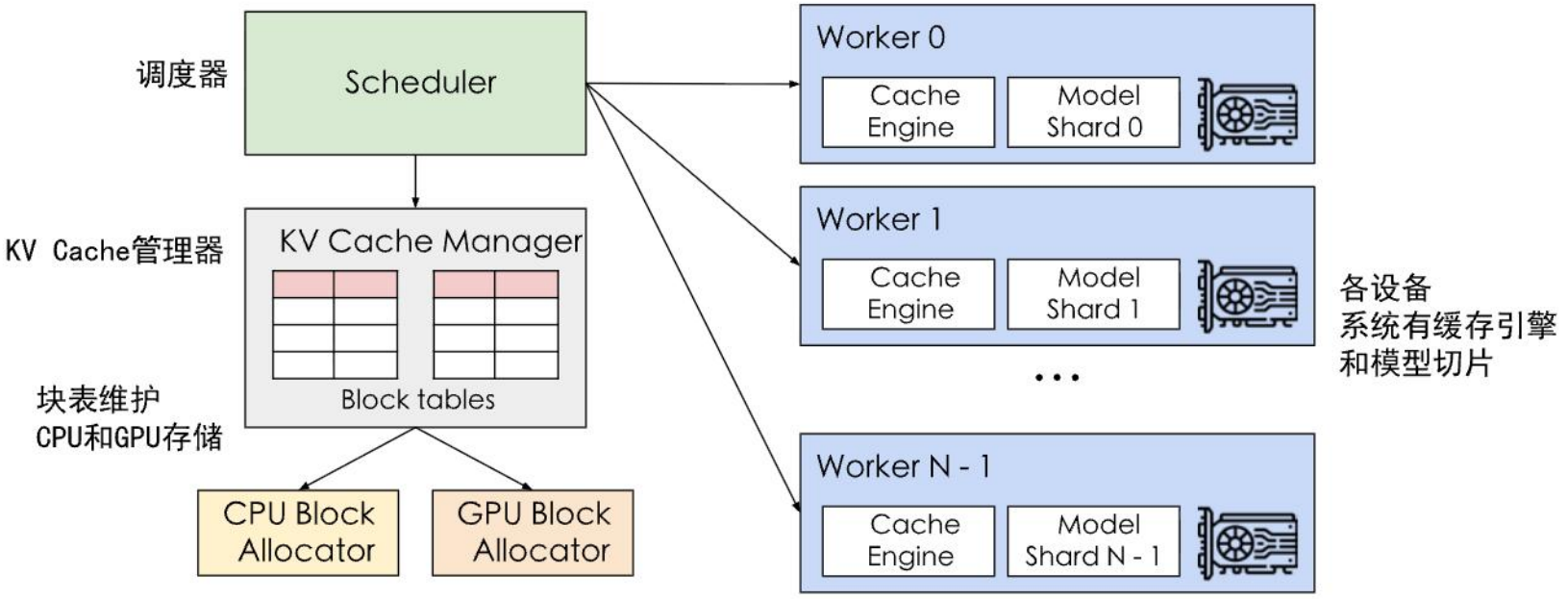


图 10.19　vLLM 逻辑架构图

调度器

决定哪些请求进入当前批次、何时抢占与释放页。

KV-Cache 管理器

通过块表维护虚拟页与物理页的映射关系。

工作器

执行单元，配备独立的缓存引擎与分片模型参数。

图源 《大语言模型：训练、精调与对齐》图 10.19

PagedAttention

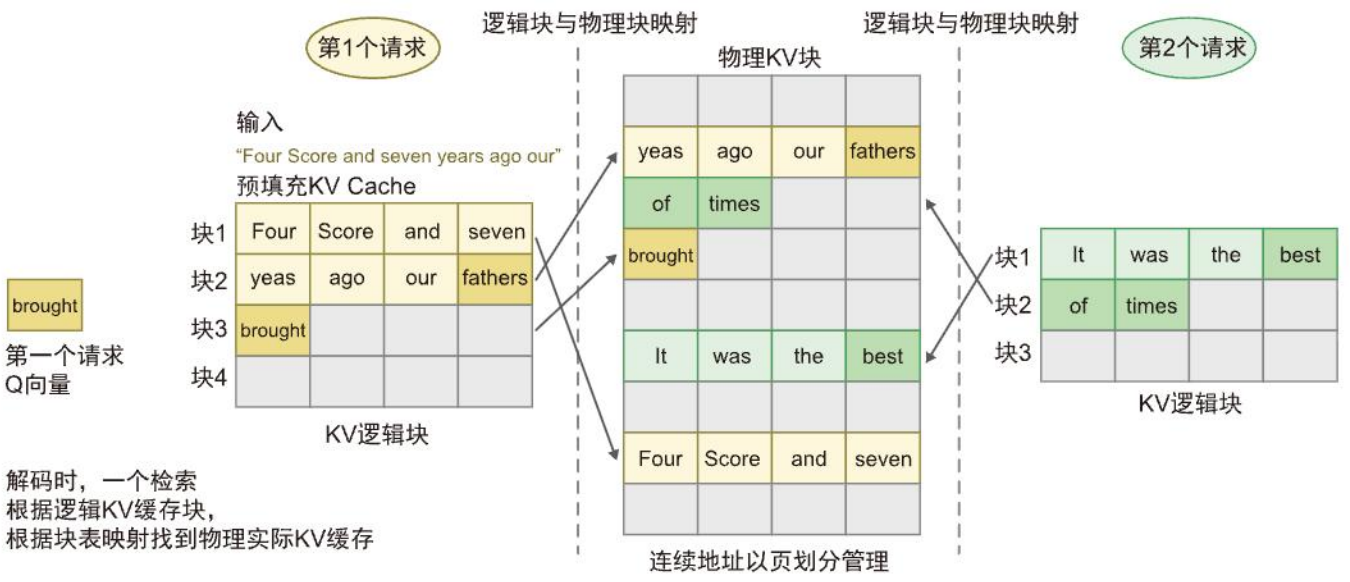


图 10.20　虚拟页管理系统处理两个请求示例

- 解码时查询向量 Q 与分页存放的 KV-Cache 计算注意力。
- 分页的 KV-Cache 与 Flash Attention 对 KV 分块处理的形式类似，可直接复用其计算内核。
- 页面填满时，块表自动建立当前页到新物理页的映射。
- 图 10.20 中请求 2 的 KV-Cache 跨越两个物理块；多请求下调页机制可灵活分配空闲页。

理论保证　分页注意力与标准注意力的计算结果一致——系统只改变了 KV-Cache 的存储管理方式。

图源　《大语言模型：训练、精调与对齐》图 10.20

静态批处理的空转

一批请求的输入长度与解码长度各不相同。部分请求提前结束解码，而其他请求仍在继续。传统批处理必须等整批全部完成，提前结束的请求所占用资源只能空转。



图 10.21(a)　批量解码

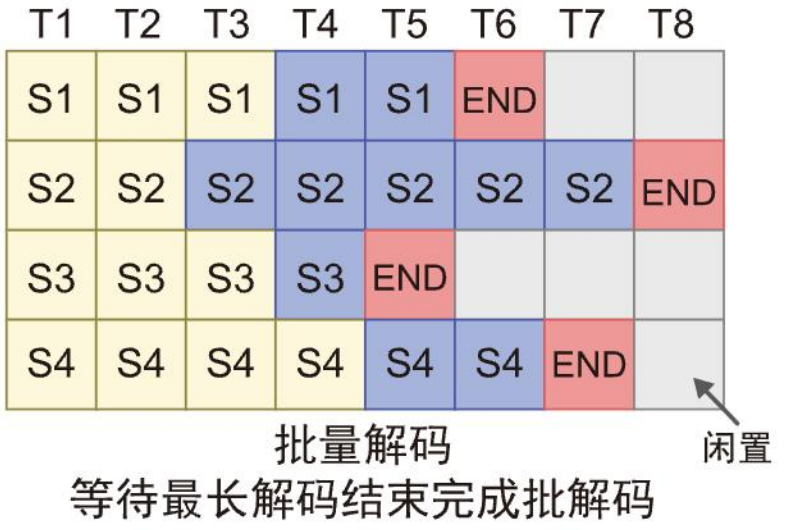


图 10.21(b)　静态批量解码

图 10.21　连续批处理示例

图 (a)：系统先预填充黄色区域的 KV-Cache，并生成首个解码词元（蓝色）。图 (b)：部分请求提前终止，静态批量解码仍需等待整批完成。

图源 《大语言模型：训练、精调与对齐》图 10.21(a)(b)

连续批处理



图 10.21(c)　连续批处理

连续批处理通过动态调度，在空闲资源上分配新请求的 KV-Cache，实现持续的词元预测。

- 终止请求释放的存储空间，立即填入新请求的 KV-Cache。
- 调度与解码持续进行，不必等待整批同步。
- 结合虚拟页管理，可同时处理更多请求。
- 遇到长序列解码时，动态资源调配避免空闲等待，实现实时响应。

图源 《大语言模型：训练、精调与对齐》图 10.21(c)

前缀共享

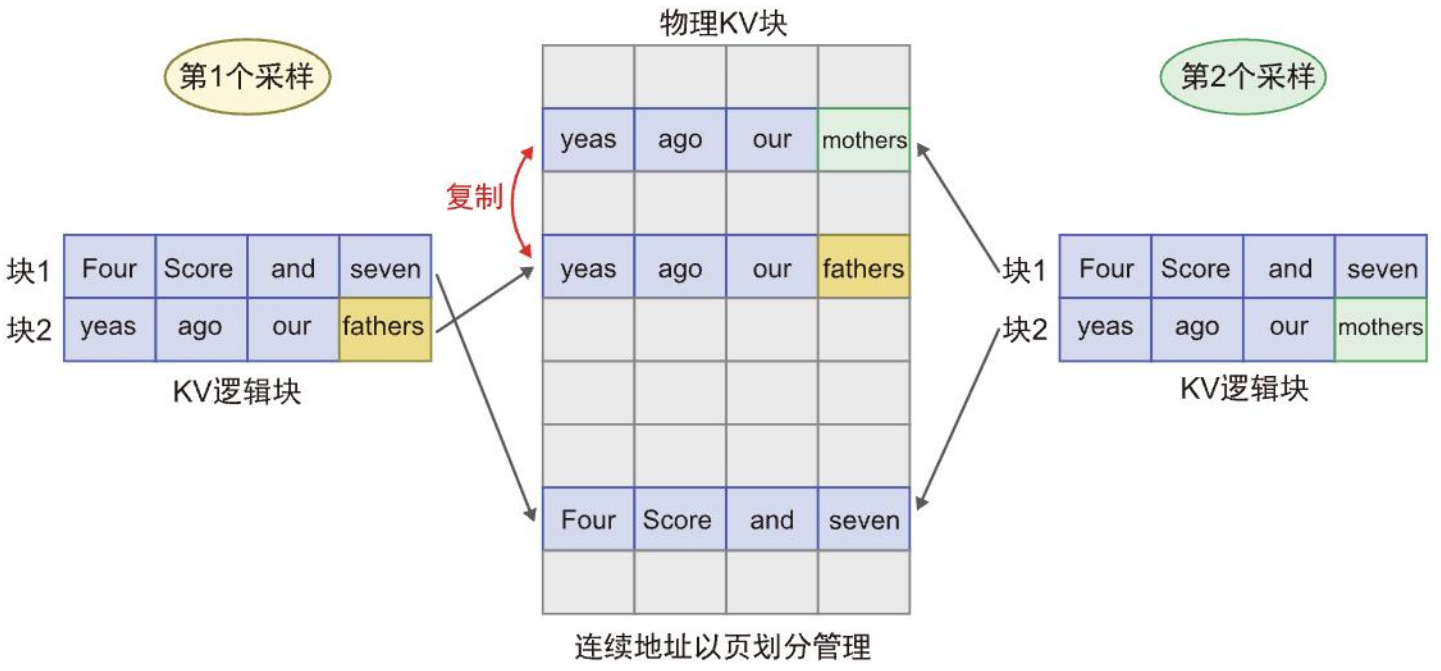


图 10.22　共享前缀解码机制

- 拒绝采样、束搜索等场景会从一个前缀分出多个解码分支。
- 分页注意力支持共享前缀：各分支共用前缀部分的物理页。
- 生成新分支时通过复制操作创建新存储块，使各分支独立解码而不干扰共享前缀。

图源 《大语言模型：训练、精调与对齐》图 10.22

束搜索中的写时复制

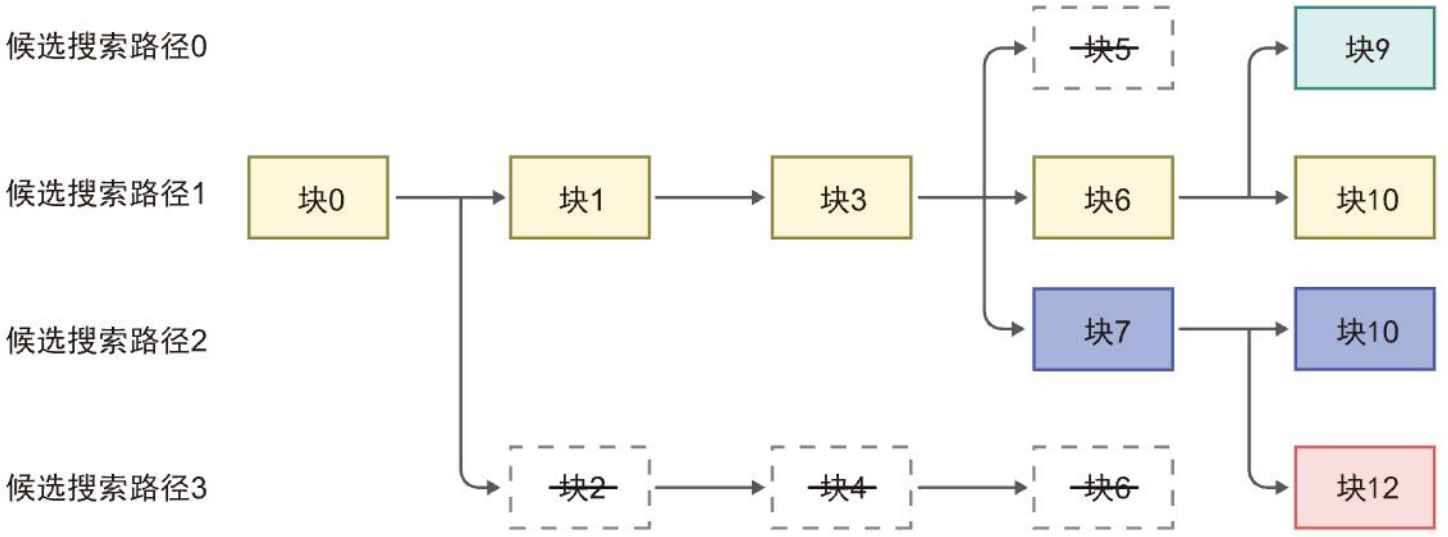


图 10.23　束搜索中的共享前缀应用

- 束搜索同时维护多条候选序列，前缀高度重复。
- 共享前缀把重复部分压缩成一份物理页，只在新分支处分叉。
- 长文本推理中还可共享系统提示词或上下文提示词的 KV-Cache，避免重复预填充。

> **收益**　显存开销显著降低，同时省掉重复的预填充计算。

图源 《大语言模型：训练、精调与对齐》图 10.23

vLLM 的三项技术

分页注意力

消除碎片

物理页 + 块表，请求可跨非连续块；利用率从 38.2% 提升到接近零浪费。

连续批处理

提高批量

请求结束即刻释放页并接纳新请求，解码不再等整批同步。

共享前缀

减少冗余

公共前缀只存一份，分支以复制创建新块，独立推进。

- 三者叠加，使解码批量与吞吐量同时提升，而注意力的数学结果保持不变。
- 分页 KV-Cache 的分块形式与 Flash Attention 的 KV 分块一致，计算内核可以直接复用。

10.4

量化压缩部署

本节内容

10.4.1 SmoothQuant

10.4.2 AWQ

权重好量化，激活值难量化。两种方法各自动刀：一个用等价变换把难度搬给权重，一个用搜索找出该保护的通道。

量化的精度格式

大语言模型参数量极大，推理时通常以 Float16 或 BFloat16 精度加载。量化能在更低比特下存储与计算。

权重 8 位 / 激活 8 位

W8A8

权重与激活值都量化，可用低比特矩阵乘。SmoothQuant 支持这一形式。

权重 4 位 / 激活 16 位

W4A16

只量化权重，适配不支持 W8A8 算子的硬件，适合端侧部署。

权重 4 位 / 激活 8 位

W4A8

权重压得更狠，激活保留 8 位，兼顾显存与精度。

- 按数据类型不同，量化方案可表示为 W8A8、W4A8、W4A16 等形式。
- 其他方法还包括 GPTQ、LLM-int4 等。本章重点读 SmoothQuant 与 AWQ。

激活值的离群通道

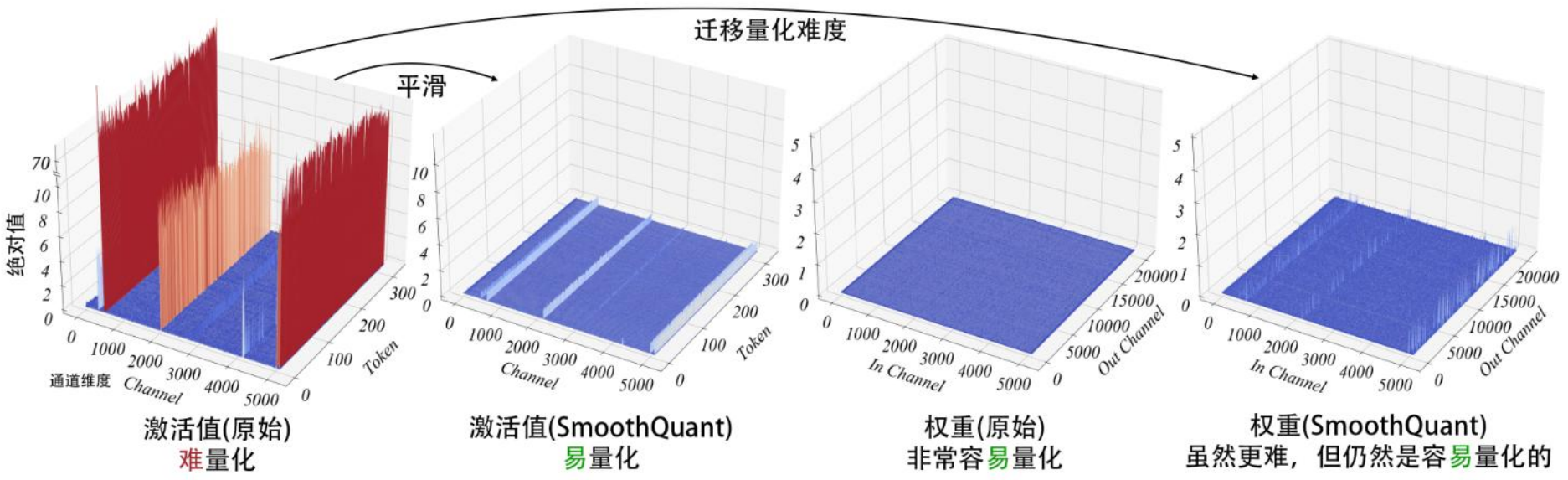


图 10.24　激活值的特征通道离群值分析图

权重均匀

权重分布通常较均匀，易于量化。

激活离群

激活值在特定特征通道上出现显著峰值（离群点）。

尺度难选

离群点增大了数值方差，使以张量块为单位的量化难以选出一个合适的缩放尺度。

SmoothQuant 是一种无须训练、保持精度的量化算法，支持对权重和激活值同时量化。

图源 《大语言模型：训练、精调与对齐》图 10.24

平滑量化

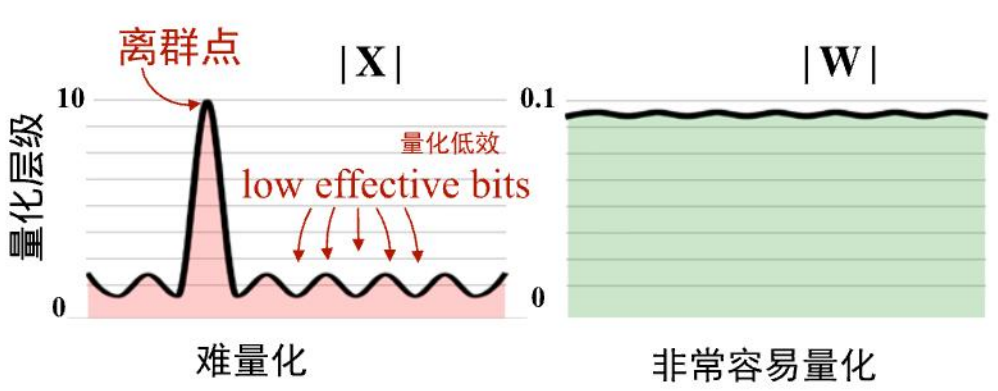


图 10.25(a)　原始

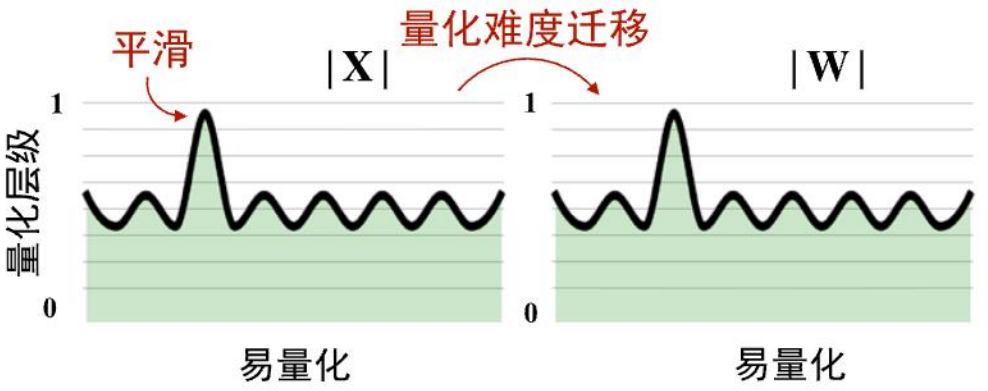


图 10.25(b)　SmoothQuant

图 10.25　SmoothQuant 量化示意图

- 对激活值做平滑，同时对权重做反向缩放，保持输出等价。
- 平滑后离群点的影响降低，量化难度显著减小。
- 本质是把激活值的量化复杂度迁移到权重，而权重量化对此类迁移不敏感。

图源 《大语言模型：训练、精调与对齐》图 10.25(a)(b)

按特征通道缩放

$$\boldsymbol{Y} = (\boldsymbol{X}\mathrm{diag}(\boldsymbol{s})^{-1}) \cdot (\mathrm{diag}(\boldsymbol{s})\boldsymbol{W}) = \hat{\boldsymbol{X}}\hat{\boldsymbol{W}} \tag{10-91}$$

$\boldsymbol{s} \in \mathbb{R}^d$ 为自适应缩放因子，d 为特征维度。先用 $\mathrm{diag}(\cdot)$ 把 $\boldsymbol{s}$ 变成对角矩阵：$\boldsymbol{X}\mathrm{diag}(\boldsymbol{s})^{-1}$ 逐特征列缩小，$\mathrm{diag}(\boldsymbol{s})\boldsymbol{W}$ 逐特征行放大，两者相乘仍等于 $\boldsymbol{X}\boldsymbol{W}$。

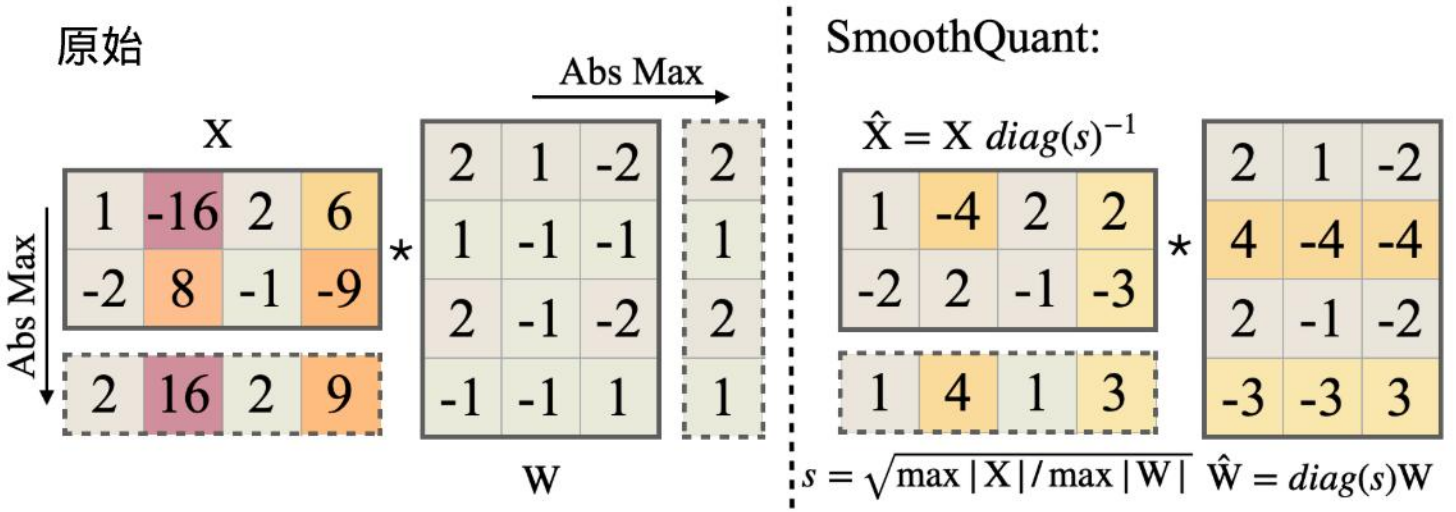


图 10.26　特征通道计算独立的缩放因子

- 量化在激活值的**特征通道维度**上执行，而非张量块级。
- （1）用 $\mathrm{diag}(\cdot)$ 把 $\boldsymbol{s}$ 转为对角矩阵，与激活值相乘即逐列缩放。
- （2）对激活值与权重分别做逆缩放与缩放，输出等价。

图源 《大语言模型：训练、精调与对齐》图 10.26；式 (10-91)

缩放因子的计算

$$\boldsymbol{s} = \sqrt{\mathrm{colmax}(\boldsymbol{X}) / \mathrm{rowmax}(\boldsymbol{W})} \tag{10-92}$$

对激活值按列（特征通道）取最大值 $\mathrm{colmax}(\boldsymbol{X})$，对参数按行取最大值 $\mathrm{rowmax}(\boldsymbol{W})$，两者相除再开方。

激活离群大

$\mathrm{colmax}(\boldsymbol{X})$ 大 → s_j 大 → 该通道被除得更多。

权重该行大

$\mathrm{rowmax}(\boldsymbol{W})$ 大 → s_j 小 → 权重不再被继续放大。

开方

把难度在激活与权重之间对半分摊。

SmoothQuant 使用校准样本从预训练数据集估计激活通道的范围。缩放因子离线统计得到，而非在推理时实时计算。

关键通道的作用

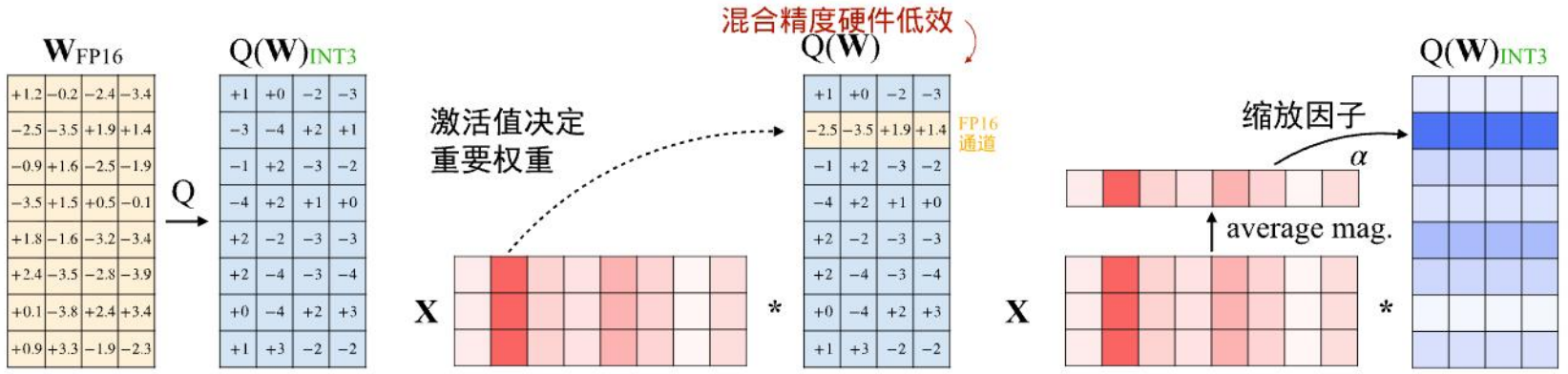


图 10.27　AWQ 量化原理图

只有少数通道敏感

权重中存在对量化精度敏感的**重要通道**；仅对约 1% 的关键通道保持原始精度，即可显著提升整体量化精度。

因此选择低比特

少量通道的数值对量化误差极为敏感，AWQ 因此专注 3-bit、4-bit 的低比特方案。

> **部署收益**　部分硬件不支持 W8A8 矩阵乘，W4A16（权重 int4、激活 fp16）可适配更多平台；低比特权重量化同时显著降低存储与计算需求，适合资源受限的设备端。

图源 《大语言模型：训练、精调与对齐》图 10.27

平滑量化的三个步骤

1　缩放权重

$\mathrm{diag}(\boldsymbol{s})\boldsymbol{W}$

→

2　量化权重

$Q(\mathrm{diag}(\boldsymbol{s})\boldsymbol{W})$

→

3　逆缩放激活

$\boldsymbol{X}\mathrm{diag}(\boldsymbol{s})^{-1}$

$$\mathcal{L}(\boldsymbol{s}) = \left\| \left(\mathrm{diag}(\boldsymbol{s})^{-1}\boldsymbol{X}\right)\left(Q(\boldsymbol{W}\mathrm{diag}(\boldsymbol{s}))\right) - \boldsymbol{X}\boldsymbol{W} \right\| \tag{10-93}$$

为保持原始输出精度，最小化“量化权重 × 缩放激活”与原始输出 $\boldsymbol{X}\boldsymbol{W}$ 的均方误差。$Q(\cdot)$ 为包含就近舍入（round-to-nearest）的整数量化函数。

$$\boldsymbol{s}^{*} = \arg\min_{\boldsymbol{s}} \mathcal{L}(\boldsymbol{s}) \tag{10-94}$$

目标是找到使损失最小的通道级缩放因子。由于就近舍入操作不可导，AWQ 采用网格搜索求次优解，而不是梯度下降。

依据 《大语言模型：训练、精调与对齐》第 10.4.2 节（式 (10-93)、(10-94)）

两种量化思路的对比

对比项	SmoothQuant	AWQ
量化对象	权重 + 激活值（W8A8）	主要量化权重（W4A16 等）
核心观察	激活值存在通道级离群点	约 1% 通道对量化误差极敏感
缩放因子来源	离线统计：$\sqrt{\mathrm{colmax}(\boldsymbol{X})/\mathrm{rowmax}(\boldsymbol{W})}$	优化求解：最小化量化前后输出误差
求解方式	闭式公式，无需训练	网格搜索（舍入不可导）
主要收益	激活值也可 8 位，启用低比特矩阵乘	权重压到 4-bit，适配更多硬件

共同点 两者都用"缩放 + 逆缩放"的等价变换：不改变数学结果，只改变数值分布，使量化误差变小。

10.5

投机解码

本节内容

10.5.1 块解码

10.5.2 投机解码

解码一次只能出一个词元。若先用低成本方式猜出若干候选，再让目标模型一次并行验证，串行步数就能成倍减少。

并行解码的思路

串行解码每次只能生成单个词元。若能一次生成多个，解码效率即可显著提升——这类技术统称并行解码。

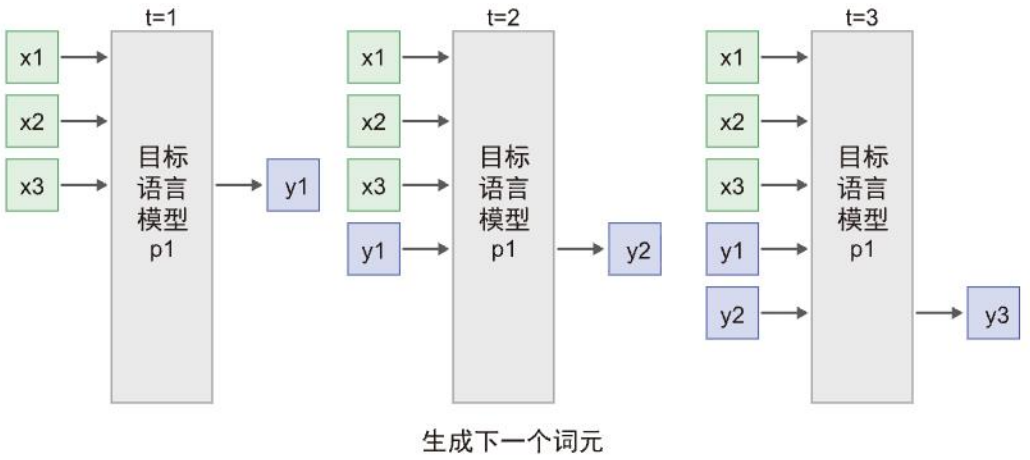


图 10.28(a)　草稿阶段：逐个生成词元

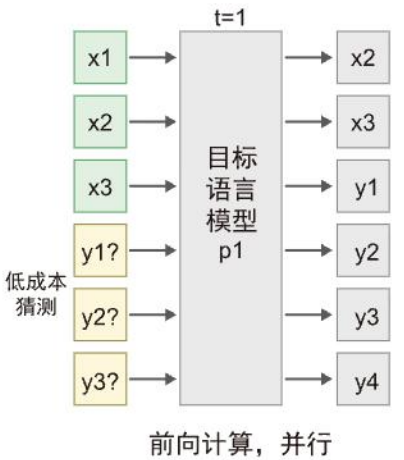


图 10.28(b)　验证阶段：并行验证候选

图 10.28　投机解码猜测与验证示意图

核心思想：先低成本猜测后续词元（黄色），再在目标模型上并行前向验证。

图源 《大语言模型：训练、精调与对齐》图 10.28(a)(b)

双模型推测架构

块并行解码采用双模型架构：目标解码模型 θ_1 预测下一词元，推测模型 θ_i 预测后续第 i 个词元。

$$\hat{y}_{j+i} = \arg\max_{y_{j+i}} p_{\theta_i}(y_{j+i} \mid \hat{\boldsymbol{y}}_{\leqslant j}, \boldsymbol{x})$$

预测阶段：推测模型 θ_i 在给定已有前缀 $\hat{\boldsymbol{y}}_{\leqslant j}$ 与输入 $\boldsymbol{x}$ 的条件下，给出位置 $j+i$ 的词元，$i = 1, 2, \cdots, k$。

$$\hat{y}_{j+i} = \arg\max_{y_{j+i}} p_{\theta_1}(y_{j+i} \mid \hat{\boldsymbol{y}}_{\leqslant j+i-1}, \boldsymbol{x})$$

验证阶段：目标模型 θ_1 以“到前一位置为止”的前缀为条件，逐位置检验预测是否正确。

预测 每个推测模型各管一个位置，互不依赖，因此可以并行生成整块候选。

验证 前 $\hat{k}$ 个词元逐个比对，通过即接受，第一个不符处终止；验证过程本身也是并行的。

目标模型 θ_1

给出真实分布，负责验证与最终采样。

推测模型 θ_i

低成本给出候选块，i 越大负责的位置越靠后。

接受

通过验证的词元作为本轮输出，失败则退回单步解码。

依据 《大语言模型：训练、精调与对齐》第 10.5.1 节（块解码流程）

预测阶段

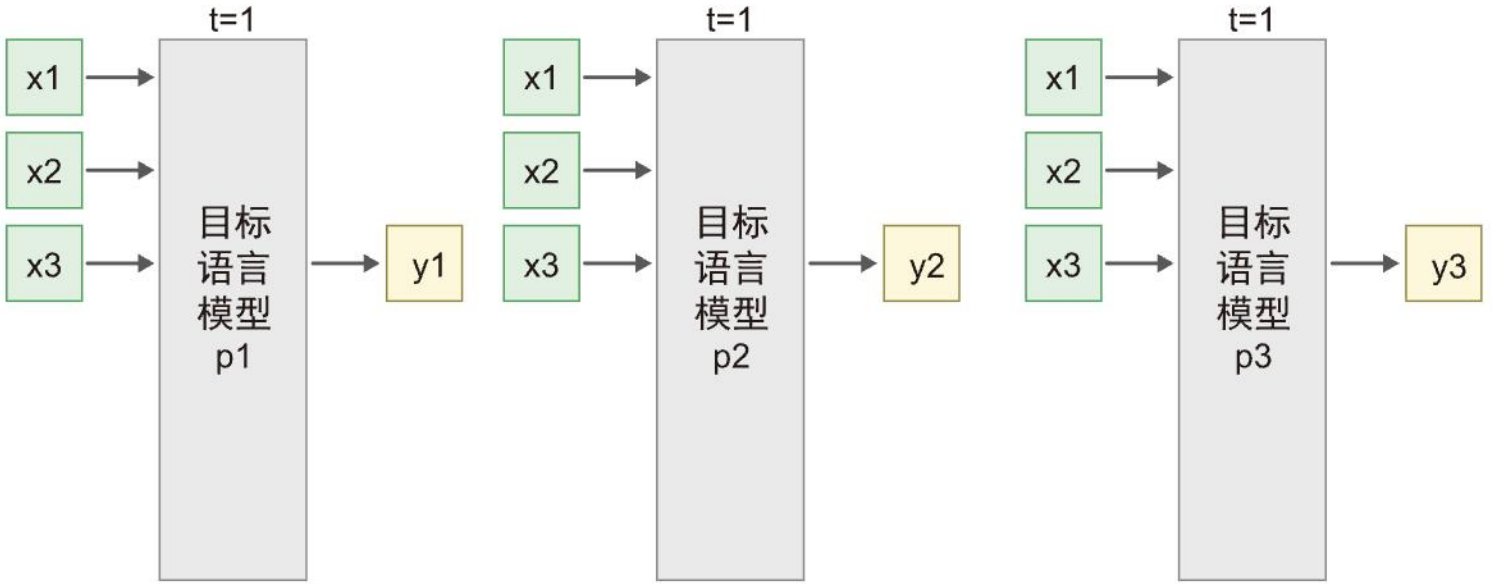


图 10.29　块并行解码流程示例预测阶段

- 第 i 个推测模型负责位置 $j+i$，各自独立。
- 同一轮内所有推测可并行执行，不必串行等待。
- 得到的是一条长度为 k 的候选块，而不是单个词元。

图源 《大语言模型：训练、精调与对齐》图 10.29

验证与接受

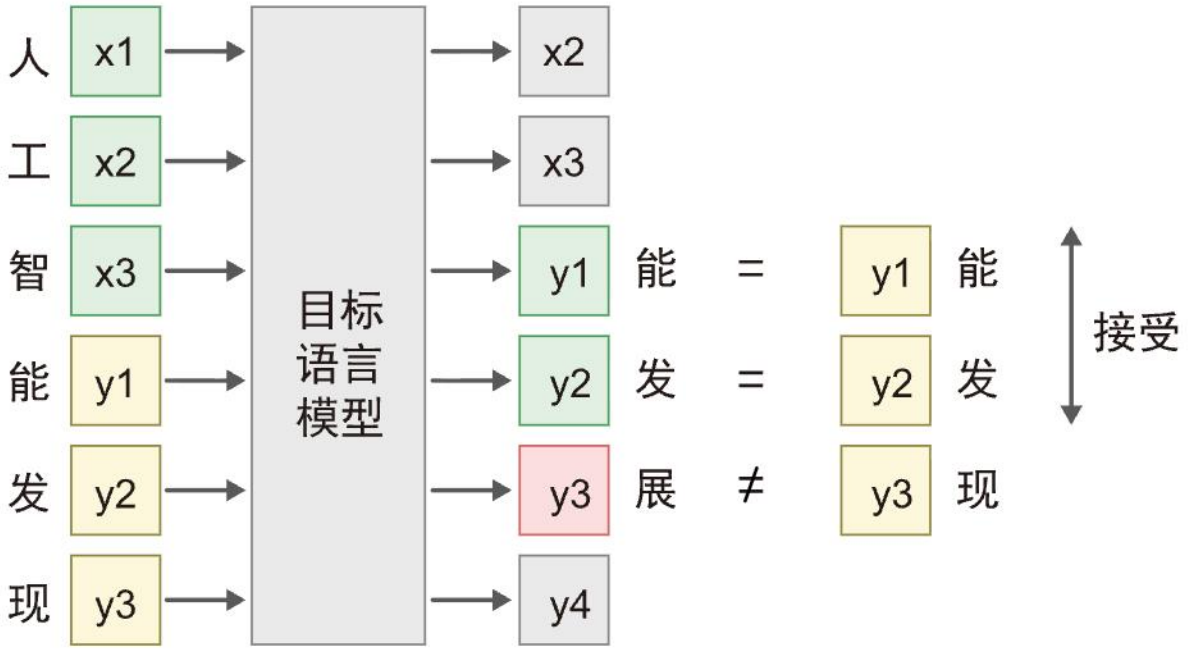


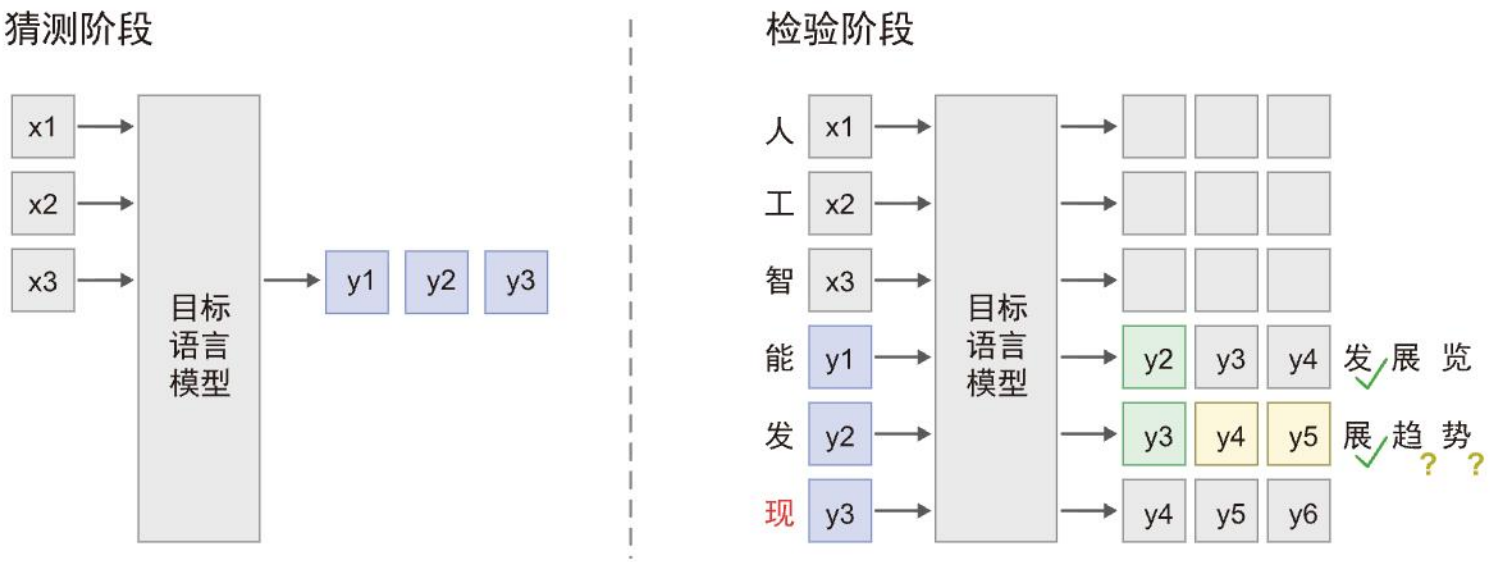
图 10.30　块并行解码流程示例验证阶段

- 验证失败时（如词元“现”），终止当前验证序列。
- 接受“能”“发”两个词元后，由 θ_1 生成新词元“展”。
- 单次迭代共解码 3 个词元。
- 若全部推测失败，系统退化为目标模型的单步解码。

接受 $\hat{k}$ 个词元后，θ_1 可继续预测位置 $y_{j+\hat{k}+1}$，进入下一轮。

图源 《大语言模型：训练、精调与对齐》图 10.30

验证与预测的并行



- 目标模型 p_{θ_1} 验证当前候选词元时，推测模型 p_{θ_i} 同步预测后续词元。
- 验证当前块（如 3 个词元）的同时生成下一候选块。
- 即使“现”验证失败，系统仍接受“展”；同步预测的“趋”“势”可直接用于下一验证周期。

图 10.31　边验证边解码示例图

> **收益**　验证与预测并行，失败也不再空等一轮，单次迭代的有效产出更高。

图源　《大语言模型：训练、精调与对齐》图 10.31

多输出头架构

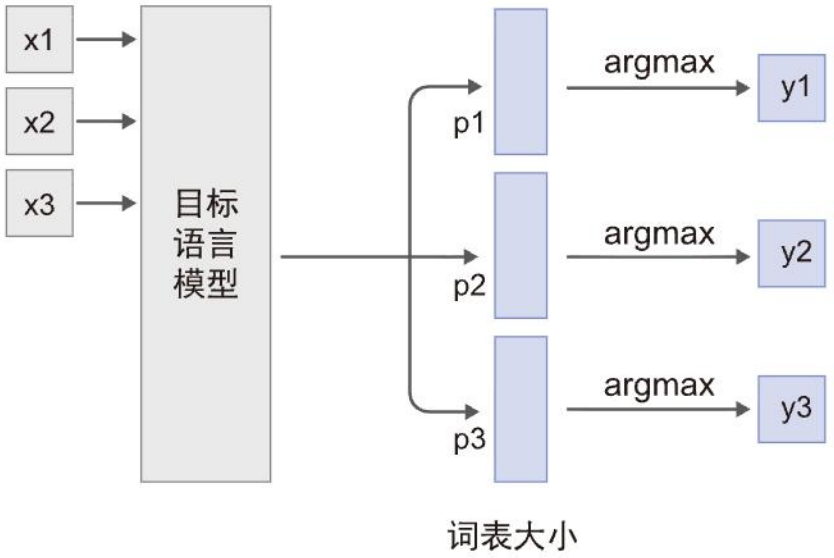


图 10.32　多输出头语言模型

验证时若要同步运行多个推测模型，开销不小。一种高效实现是把 Transformer 的单输出头改为多输出头结构。

- 第 i 个头负责预测后续第 i 个词元；$i = 1$ 时即常规的下词元预测。
- 各头具有差异化训练目标，但共享主干参数。
- 无需额外执行推测模型，单模型即可完成验证-预测并行，多头共享参数也提升计算效率。

Medusa、Eagle 系列等算法通过优化推测机制，显著提升了候选词元的接受率。

图源 《大语言模型：训练、精调与对齐》图 10.32

草稿模型与目标模型

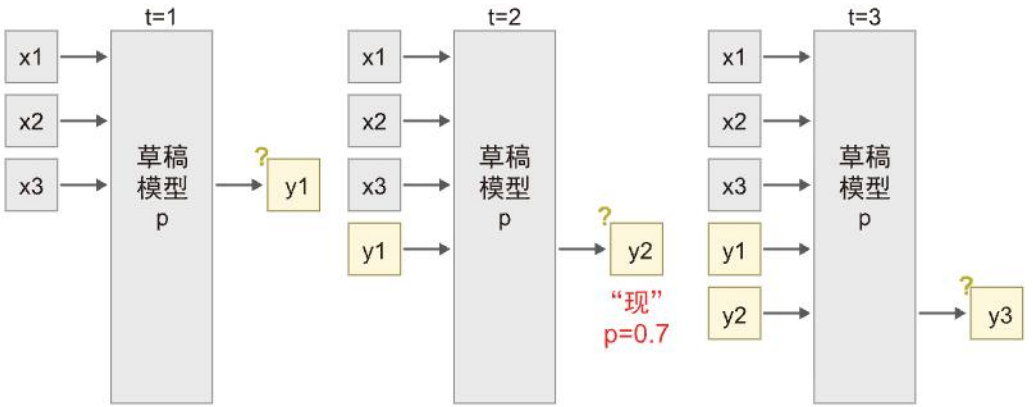


图 10.33(a)　草稿阶段

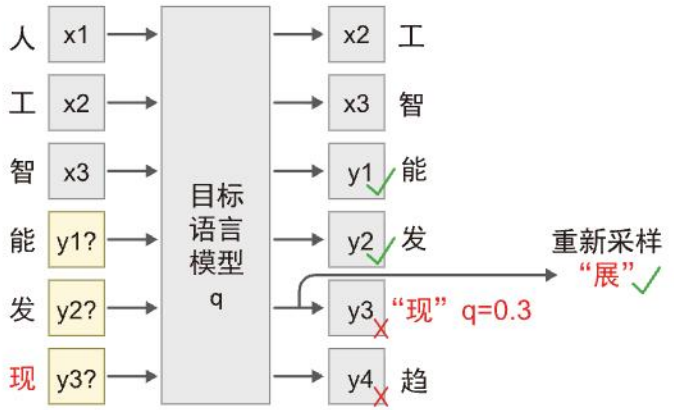


图 10.33(b)　验证阶段

图 10.33　投机解码示例

- 草稿模型尺寸更小、解码更快，输出分布接近目标模型，可通过知识蒸馏学习目标模型的分布。
- 草稿阶段一次预测 3 个词元；验证阶段目标模型逐位检验。
- “现”未通过检验，系统重采样得到“展”。

图源 《大语言模型：训练、精调与对齐》图 10.33(a)(b)

投机解码流程

$$\tilde{x}_t \sim p(x|\boldsymbol{x}, \tilde{\boldsymbol{x}}_{<t}), \qquad t = 1, 2, \cdots, k \tag{10-95}$$

第 1 步 草稿模型 p 自回归地解码出 k 个候选词元 $\tilde{\boldsymbol{x}} = (\tilde{x}_1, \cdots, \tilde{x}_k)$。

$$q(x|\boldsymbol{x}, \tilde{x}_t), \qquad t = 1, 2, \cdots, k \tag{10-96}$$

第 2 步 输入 $\boldsymbol{x}$ 与候选 $\tilde{\boldsymbol{x}}$ 拼接后送入目标模型 q，一次前向并行算出各位置的条件概率。

$$r < \min\left(1, \frac{q(x|\boldsymbol{x}, \tilde{\boldsymbol{x}}_{<t})}{p(x|\boldsymbol{x}, \tilde{\boldsymbol{x}}_{<t})}\right) \qquad t = 1, 2, \cdots, k \tag{10-97}$$

第 3 步 采样 $r \sim U[0,1]$；不等式成立则接受 x_t。比值 q/p 越大越容易接受。

$$\tilde{x}_t \sim (q(x|\boldsymbol{x}, \tilde{\boldsymbol{x}}_{<t}) - p(x|\boldsymbol{x}, \tilde{\boldsymbol{x}}_{<t})) \tag{10-98}$$

第 4 步 · 拒绝时 从两分布之差（取正部并归一化）重采样，并终止后续验证。

$$x_{k+1} \sim q(x|\boldsymbol{x}, \tilde{\boldsymbol{x}}) \tag{10-99}$$

第 5 步 · 全部接受时 由目标模型再采样一个词元，本轮推进 $k+1$ 个词元。

- 全部接受：$\boldsymbol{x} \leftarrow (\boldsymbol{x}, \tilde{\boldsymbol{x}}, x_{k+1})$；第 t 个被拒：$\boldsymbol{x} \leftarrow (\boldsymbol{x}, \tilde{\boldsymbol{x}}_{\leqslant t})$。
- 出现终止词元或达到目标长度则退出，否则回到第 1 步。
- 草稿阶段是 k 次串行小模型前向，验证阶段是 1 次大模型前向；一轮最少产出 1 个词元、最多 $k+1$ 个。

依据 《大语言模型：训练、精调与对齐》第 10.5.2 节（式 (10-95)–(10-99)）

等价性证明

$$\Pr(X=x)=\Pr(\tilde{x}=x)\Pr(\text{接受}|\tilde{x}=x)+\Pr(\text{拒绝})\Pr(X=x|\text{拒绝}) \tag{10-100}$$

$$\Pr(\tilde{x}=x)\Pr(\text{接受}|\tilde{x}=x)=p(x)\min\left(1,\tfrac{q(x)}{p(x)}\right) \tag{10-101}$$

$$=\min\left(p(x),q(x)\right) \tag{10-102}$$

$$\Pr(X=x|\tilde{x}\text{ 拒绝})=(q(x)-p(x))_+ \tag{10-103}$$

$$\Pr(\tilde{x}\text{ 拒绝})=1-\Pr(\tilde{x}\text{ 接受}) \tag{10-104}$$

$$=1-\sum_{x'}\min(p(x'),q(x')) \tag{10-105}$$

$$=\sum_{x'}\max(0,q(x')-p(x')) \tag{10-106}$$

$$\Pr(\tilde{x}\text{ 拒绝})\Pr(X=x|\tilde{x}\text{ 拒绝})=\max(0,q(x)-p(x)) \tag{10-107}$$

$$\Pr(X=x)=\min(p(x),q(x))+\max(0,q(x)-p(x)) \tag{10-108}$$

$$=q(x) \tag{10-109}$$

式 (10-100) 按“采到并被接受”“被拒后重采样”两种互斥情形拆开；式 (10-101) 代入接受概率 $\min(1,q/p)$，式 (10-102) 合并为 $\min(p,q)$；式 (10-103) 给出重采样的条件分布（取差的正部并归一化）；式 (10-105) 把接受事件按 x' 拆开求和，式 (10-106) 用 $1-\min(a,b)=\max(0,b-a)$；式 (10-107) 中归一化因子与式 (10-106) 的求和相同，相乘即约去；式 (10-108) 合流后两种情形都得到 $q(x)$。

结论 投机解码的输出分布与目标模型 q 完全一致，因此是“无损加速”，而不是用精度换速度。

块解码与投机解码

对比项	块并行解码	投机解码
候选来源	多个推测模型或多个输出头	一个更小的草稿模型
模型数量	θ_1 加 θ_i，或单模型多头	草稿模型 p + 目标模型 q
验证方式	θ_1 逐位置比对，遇错截断	按 q/p 的比值做随机接受检验
输出保证	接受即输出，拒绝则截断	分布与 q 严格等价
关键指标	接受长度 $\hat{k}$	接受率与草稿成本

两者的共同前提：验证时一次前向可以并行算出多个位置的条件概率。这正是 Transformer 训练阶段就具备的并行能力，在解码时被重新利用起来。

依据 《大语言模型：训练、精调与对齐》第 10.5.1、10.5.2 节

10.6

小结

四种方法分别盯住访存、显存管理、数值精度与解码步数；它们都建立在同一个前提上——自回归解码必须一次一步。

本节内容

10.6 小结

思考题与实践题

推理加速的四条路线

方法	看到的瓶颈	核心手段
Flash Attention	HBM 与 SRAM 间搬运	借 Online-Softmax 做分块计算，中间矩阵不落地
vLLM	KV-Cache 利用率低	分页注意力 + 连续批处理，提升批处理量
SmoothQuant / AWQ	激活值分布难量化	等价变换做平滑，使量化结果逼近原始输出
并行解码	解码串行、一次一词元	把串行解码问题转化为并行验证问题

- 语言模型加速还包括剪枝、蒸馏、稀疏化、局部注意力（如窗口注意力）等方法。
- 系统层面可设计预填充与解码分离的架构；对 MoE 架构与 RNN 类模型，也有各自的加速技术。

推理加速的主线

不重复搬

注意力分块，$\boldsymbol{S}$ 与 $\boldsymbol{P}$ 不再写回 HBM，只留 $O(N)$ 的状态。

不重复占

KV-Cache 按页分配，请求结束即释放；公共前缀只存一份。

不重复存

权重与激活值降到 8 位或 4 位，同样的显存装下更大的模型与批量。

不重复算

一次前向并行验证多个候选词元，把 k 步串行换成 1 步并行。

四种手段可以叠加：分页 KV-Cache 复用 Flash Attention 的内核；投机解码的验证阶段本身就是一次并行前向。

思考题与实践题

1 预填充与解码的计算特性有何不同？分别对应哪个性能指标？

2 式 (10-20) 中 $\tilde{l} \neq l_1 + l_2$，请说明差在哪里，并写出正确的合并式。

3 Flash Attention 的复杂度仍是 $O(N^2 d)$，为什么反而更快？省下的是哪一部分开销？

4 V2 把 $\mathrm{diag}(\boldsymbol{L}_i)^{-1}$ 移出内循环，为什么这不改变最终结果？

5 分页注意力为什么能与标准注意力保持数学等价？块表在其中起什么作用？

6 SmoothQuant 的缩放因子为何取几何平均？若只对激活值缩放会怎样？

7 投机解码在全部候选都被拒绝时，输出分布为何仍等于目标分布？

8 实现 Flash Attention V2 的前向与反向算法。

9 实现分页注意力。

10 在块解码基础上进行优化，使得各头输出之间有前后依赖。

11 实现带分页 KV-Cache 管理器的连续批处理服务。

依据 《大语言模型：训练、精调与对齐》第 10.1–10.6 节